



SOFTWARE ENGINEERING PROJECT

EEG Analysis Platform for Prestimulus EEG-Based Behavioral Prediction

BY

**Nantawat Suksirisunt
Naytitorn Chaovirachot**

**DEPARTMENT OF COMPUTER ENGINEERING
FACULTY OF ENGINEERING
KASETSART UNIVERSITY**

Academic Year 2565

EEG Analysis Platform for Prestimulus EEG-Based Behavioral Prediction

BY

**Nantawat Suksirisunt
Naytitorn Chaovirachot**

**This Project Submitted in Partial Fulfillment of the
Requirement for Bachelor Degree of Engineering
(Software Engineering)
Department of Computer Engineering, Faculty of
Engineering KASERTSART UNIVERSITY
Academic Year 2565**

Approved By:

Advisor **Date**
(Asst. Prof.Dr. Thanawin Rakthanmanon)

Head of Department **Date**
(Assoc. Prof.Dr. Punpiti Piamsa-Nga)

Abstract

Electroencephalography (EEG) is widely used to study how neural activity relates to behavior, but practical EEG research often depends on ad-hoc exploratory workflows. Interactive notebooks are convenient for inspection and rapid iteration; however, hidden execution state, environment dependency, and inconsistent preprocessing make it difficult to reproduce results, compare experiments systematically, and share a workflow that others can run in the same way.

This project investigates whether prestimulus EEG can predict behavioral performance in the Healthy Brain Network EEG (HBN-EEG) dataset. We focus on the Contrast Change Detection (CCD) task and use the 2-second inter-trial interval (ITI) segment as a constrained prestimulus window for modeling.

The proposed approach extracts steady-state and spectral features from CCD-ITI epochs after standard preprocessing (e.g., bandpass filtering, artifact mitigation, and normalization). We train models to predict reaction time (RT) as a regression target and trial-level correctness (accuracy) as a binary classification target. Performance is evaluated using regression metrics for RT (e.g., MAE/MSE) and classification metrics for correctness (accuracy, F1-score, precision, and recall).

To support reproducible experimentation, the software component provides a lightweight interface for configuring preprocessing, running feature extraction and modeling, and producing standardized outputs (tables/plots) without relying on notebook-based trial-and-error.

Keywords: Prestimulus EEG, HBN-EEG, Contrast Change Detection, Reaction Time, Correctness Prediction, Inter-trial Interval, EEG Preprocessing

Acknowledgement

We would like to thank the Department of Computer Engineering, Faculty of Engineering, Kasetsart University, for continuous academic support, and express our sincere gratitude to our advisor, Asst. Prof.Dr. Thanawin Rakthanmanon, for invaluable guidance, constructive feedback, and continuous encouragement throughout this project. Their support helped us shape both the technical direction and the research presentation of this work.

We would like to express our sincere appreciation to the BRAIN-Interfaces Lab, under the School of Information Science and Technology (IST) at Vidyasirimedhi Institute of Science and Technology (VISTEC), for providing essential computational resources that were critical to dataset processing and model training. We also extend our deep gratitude to Assoc. Prof. Dr. Theerawit Wilaiprasitporn for valuable recommendations and dedicated mentorship throughout this work. In addition, we gratefully acknowledge the senior members of the BRAIN-Interfaces Lab for their practical advice, continuous support, and substantial technical guidance.

In addition, we acknowledge the EEG Foundation Challenge and the HBN-EEG data providers for making high-quality EEG resources available for research.

Finally, we are deeply thankful to our teammates, friends, and families for their patience, motivation, and constant encouragement throughout this journey.

Nantawat Suksirisunt
Naytitorn Chaovirachot

Table of Content

Abstract	i
Acknowledgement	ii
Table of Table	viii
Table of Figure	x
Chapter 1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Solution Overview	3
1.4 Target User	3
1.5 Benefit	4
1.6 Scope of the Study	4
1.7 Terminology	5
1.8 Personal Development Plan	5
Chapter 2 Literature Review and Related Work	7
2.1 Competitor Analysis	7
2.2 Literature Review	8
2.2.1 Foundational Works Leveraged	8
2.2.2 Key Insights and Adaptations from Recent Methodologies	8
2.3 Positioning of This Work	9
2.3.1 CCD Experimental Design	9
Chapter 3 Requirement Analysis	11
3.1 Stakeholder Analysis	11

3.2	Users and Interaction Modes	12
3.3	User Stories	12
3.4	Use Case Overview	13
3.5	Use Case Model	13
3.5.1	Use Case 0: Configure Data Root	13
3.5.2	Use Case 1: Inspect and Verify a Task Recording	14
3.5.3	Use Case 2: Build CCD Epochs and Labels (Pres- timulus/ITI)	14
3.5.4	Use Case 3: Build Training Datasets	15
3.5.5	Use Case 4: Train/Test Models and Export Artifacts	16
3.6	User Interface Design	16
3.7	Target and Development System	18
3.7.1	Data Handling and Input Capabilities	18
3.7.2	Core Processing and Modeling Requirements	18
3.7.3	User Interface and Visualization Needs	18
3.7.4	Non-Functional Requirements	19
3.8	Functional Requirements	19
3.9	Non-Functional Requirements	21
3.10	Extensibility (Easy to Extend)	21
3.11	Traceability (Stories to Requirements)	23
Chapter 4	Software Architecture Design	24
4.1	Software Architecture	24
4.2	Domain Model	24
4.3	Design Patterns	26
4.4	Design Class Diagram	27
4.5	Sequence Diagram	32
4.5.1	Experiment Execution Flow	32
4.6	Pipeline	33
4.6.1	AI Processing Pipeline	33
Chapter 5	AI Component Design	35
5.1	Business Context and AI Integration	36
5.2	Goal Hierarchy	38
5.3	Task Requirements Analysis Using AI Canvas	39

5.3.1	AI Task Requirements	39
5.3.2	AI Canvas Summary	40
5.3.3	Innovation	40
5.4	AI Model Development and Evaluation	40
5.4.1	Model Development and Training Strategy	40
5.4.2	Testing and Validation Approach	40
5.4.3	Performance Results and Justification	41
5.5	User Experience Design with AI	43
5.6	Deployment Strategy	43
5.7	Proof of Concept (Service Demonstration)	44
Chapter 6	Software Development	45
6.1	Software Development Methodology	45
6.2	Technology Stack	46
6.3	Coding Standards	47
6.4	Progress Tracking Report	48
6.5	Final Iteration Status	50
Chapter 7	Deliverable	51
7.1	Software Solution	51
7.1.1	System Overview	51
7.1.2	Input Configuration	52
7.1.3	Mode-Based Interaction	52
7.1.4	Action Selection	53
7.1.5	Execution and Logging	54
7.1.6	System Characteristics	55
7.2	Test Report	56
7.2.1	Testing Approach	56
7.2.2	Functional Testing	56
7.2.3	Reproducibility Testing	58
7.2.4	Machine Learning Evaluation Testing	58
7.2.5	Classification Runs	59
7.2.6	Regression Runs	59

Chapter 8	Project Defense and Project Expo	79
8.1	First Project Defense	79
8.1.1	Comments and Feedback from the Evaluators	79
8.1.2	Revisions Required, Actions Taken, and Outcomes	80
8.2	Second Project Defense	80
8.2.1	Comments and Feedback from the Evaluators	80
8.2.2	Additional Revisions, Actions Taken, and Final Outcomes	81
8.2.3	Suggestions on How Project Defense Can Better Support Software Development	82
8.3	Project Expo Participation	82
8.3.1	Poster Presentation and Poster Components	82
8.3.2	Event Photos and Awards Received	85
8.3.3	Questions Raised During the Expo and the Team's Responses	87
8.3.4	Recommendations for Improving the Expo	87
Chapter 9	Knowledge and Skills Evaluation	89
9.1	Comparison Between Program Targets and AI-Based Evaluation	89
9.1.1	Executive Summary	89
9.1.2	Detailed Evaluation Table	90
9.1.3	Justification for Each Dimension	90
9.1.4	Gap Analysis	93
9.1.5	AI Final Conclusion	93
9.1.6	Radar Chart Comparing Program Targets and AI Evaluation	94
9.1.7	Reflection on the AI Evaluation	95
9.2	Self-Evaluation by the Student	96
9.2.1	Radar Chart Comparing Program Targets, AI Evaluation, and Self-Evaluation	96
9.2.2	Reflection on Self-Evaluation Compared with AI Evaluation	97
9.3	Advisor Evaluation	98

9.3.1	Radar Chart Comparing Program Targets, AI Evaluation, Self-Evaluation, and Advisor Evaluation	98
9.3.2	Advisor's Comments on the Evaluation	99
9.3.3	Student Reflection on the Advisor's Evaluation	99
Chapter 10	Conclusion and Discussion	100
10.1	Reflection	100
10.2	Challenges	100
10.3	Future Work	101
Appendix A:	Reproducibility Summary	107
Appendix B:	Detailed Result Tables	110

Table of Table

3.1	Functional requirements summary for the EEG Workbench.	20
3.2	Non-functional requirements summary.	21
3.3	Extensibility change cases showing how new capabilities can be added with minimal modification to existing code.	22
3.4	Traceability matrix from user stories to requirements.	23
6.1	Progress tracking summary for the prestimulus behavioral prediction project.	49
7.1	Core functional test cases verifying the expected operational workflow.	57
7.2	Selected classification runs from W&B export.	59
7.3	Regression runs from W&B export.	59
9.1	AI-based evaluation of knowledge and skills dimensions.	90
9.2	Gap analysis between expected curriculum levels and AI-assessed levels.	93
B.1	Detailed classification run comparison.	110
B.2	Detailed regression run comparison.	110

Table of Figure

2.1	CCD experimental setup with fixation-centered visual stimulation and left/right response mapping.	10
2.2	Example CCD trial sequence showing contrast-change events, button responses, and feedback.	10
3.1	CCD trial timeline used to define ITI-window selection before target onset, with contrast-change events, button responses, and feedback markers.	15
3.2	Initial Notebook UI Design	17
4.1	Domain Model Diagram	31
4.2	EEGKit main interaction flow (Web UI → Backend API → Pipeline → AI Trainer/Result Store)	33
5.1	Workflow for prestimulus EEG-based behavioral prediction from CCD-ITI segments.	36
6.1	Tech Stack Diagram	47
7.1	React Notebook Interface – Model visualization and execution for single subject	61
7.2	React Notebook Interface – Model visualization and execution for multi subject	62
7.3	React Notebook Interface - model visualization and execution for multiple subjects (2)	63
7.4	Sensor layout	64
7.5	Time-domain signal	65
7.6	Frequency spectrum	66
7.7	Epoch visualization	67
7.8	Evoked response	68
7.9	Evoked topomap	69
7.10	Evoked joint plot	70
7.11	Evoked per condition	71
7.12	Signal-to-noise ratio spectrum	72

7.13	Power spectral density grid	73
7.14	Signal-to-noise ratio grid	74
7.15	Evoked grid view	75
7.16	EEG Table	75
7.17	Epochs Table	76
7.18	Metadata Table	76
7.19	Build Dataset Table	76
7.20	Train EEGMultiNetRegression Table	77
7.21	Model Summary Table	78
8.1	Poster	84
8.2	Event Photo	86
9.1	Radar chart comparing expected curriculum levels and AI-assessed levels across five evaluation dimensions.	95
9.2	Radar chart comparing program targets, AI evaluation, and self-evaluation across five dimensions.	97

Chapter 1

Introduction

This chapter introduces the motivation for prestimulus EEG-based behavioral prediction and explains why the HBN-EEG CCD task is an appropriate case study. In addition to the research question, it motivates the software need: a reproducible, end-to-end workflow that reduces reliance on fragmented notebooks and makes experiments easier to rerun, compare, and report.

1.1 Background

Electroencephalography (EEG) is widely used in neuroscience and brain-computer interface (BCI) research to study brain activity with high temporal resolution. Modern EEG datasets, such as those following the Brain Imaging Data Structure (BIDS), often contain large volumes of multi-subject recordings with complex experimental conditions. Analyzing such data typically involves multiple stages, including signal inspection, preprocessing, feature extraction, and model development.

In current research practice, these workflows are commonly implemented using notebook-based environments. While flexible, this approach requires manual scripting for each step, leading to fragmented processes and limited reproducibility. Researchers must repeatedly configure preprocessing pipelines, adjust parameters, and re-run computations, which becomes increasingly inefficient as dataset size and complexity grow.

Furthermore, tasks such as comparing signals across subjects or exploring cohort-level patterns require additional manual effort. As a result, the overall workflow becomes time-consuming and computationally

expensive, often requiring significant hardware resources for repeated execution.

This work utilizes the Healthy Brain Network EEG (HBN-EEG) dataset, which provides high-density 128-channel recordings across multiple experimental tasks. The dataset is distributed in BIDS format and incorporates Hierarchical Event Descriptors (HED), supporting standardized and reproducible EEG analysis. It is provided through the EEG Foundation Challenge (EEG2025), highlighting the growing need for scalable and efficient analysis tools.¹

1.2 Problem Statement

Despite the availability of powerful EEG processing libraries, the current workflow for EEG data analysis remains inefficient and difficult to scale. The reliance on notebook-based pipelines introduces several key challenges.

First, setting up and executing preprocessing and analysis pipelines requires substantial time and effort. Even minor changes in parameters often necessitate re-running the entire computation, resulting in slow iteration cycles.

Second, existing workflows provide limited support for systematic comparison across subjects. Researchers must manually select and process individual datasets, making cohort-level analysis cumbersome and error-prone.

Third, the computational cost associated with repeated preprocessing and model experimentation can be high. This not only slows down research progress but also may require significant computational resources.

These limitations hinder efficient exploration of EEG data and slow down the development of machine learning models for tasks such as behavioral prediction

¹See [1, 2].

1.3 Solution Overview

To address these challenges, this thesis proposes an interactive EEG analysis platform designed to streamline the research workflow from data inspection to model experimentation.

The platform integrates data selection, preprocessing, visualization, and machine learning experimentation into a unified, parameter-driven workflow. By centralizing these functionalities, the system enables users to perform complex analysis tasks without relying on manual scripting.

A key design feature of the platform is the mode-action interface, which organizes functionality into four primary operational modes: Plot Mode, Grid Plot Mode, Data Mode, and AI Mode. Each mode provides context-specific actions, allowing users to focus on relevant tasks while maintaining a structured workflow.

The platform also supports flexible subject selection and cohort filtering. Users can perform detailed analysis on individual subjects or apply attribute-based filters to explore patterns across multiple participants. This capability facilitates both fine-grained inspection and large-scale exploratory analysis.

In addition, the platform provides integrated support for machine learning experimentation, including model training and basic evaluation. This allows users to iteratively refine models within the same environment used for data exploration.

Overall, the system is designed to enable more efficient iteration during EEG research by reducing reliance on repetitive notebook-based processes.

1.4 Target User

The platform is designed for researchers and students working in fields related to neuroscience, brain-computer interfaces (BCI), and EEG data analysis. These users typically require tools for inspecting neural

signals, preparing datasets, and developing machine learning models for experimental tasks.

The system is particularly beneficial for individuals who work with multi-subject EEG datasets and require efficient methods for comparing signals and conducting exploratory analysis.

1.5 Benefit

The proposed platform offers several key benefits.

First, it helps reduce the time required for EEG data analysis by minimizing repetitive manual scripting. Users can perform data selection, preprocessing, and visualization through an integrated interface, supporting faster iteration.

Second, the platform improves usability by providing a structured workflow through its mode-action design. This reduces complexity and helps users focus on specific analysis tasks.

Third, the system enhances reproducibility by maintaining a parameter-driven workflow and logging user actions. This ensures that analysis steps can be consistently replicated.

Finally, the integration of machine learning experimentation within the same environment allows users to seamlessly transition from data exploration to model development. This unified approach supports more efficient research workflows, particularly for tasks involving behavioral prediction from EEG signals.

1.6 Scope of the Study

This study focuses on the design and development of an interactive EEG analysis platform that supports data exploration, preprocessing, visualization, and machine learning experimentation within a unified workflow. The system is intended to facilitate efficient analysis of multi-subject EEG

datasets, particularly those structured in standardized formats such as BIDS.

The scope of the platform includes functionality for single-subject inspection and cohort-level analysis through flexible filtering mechanisms. It also supports the configuration and execution of preprocessing pipelines, as well as model training and basic evaluation using metrics such as training loss, validation loss, mean absolute error (MAE), and coefficient of determination (R^2).

The platform is demonstrated using the Healthy Brain Network EEG (HBN-EEG) dataset, with a focus on behavioral prediction tasks, including contrast change detection (CCD). This serves as a representative use case to validate the system's capability in supporting real-world EEG research workflows.

However, this study does not aim to develop or optimize state-of-the-art machine learning models. The performance of predictive models is not the primary focus, and the platform is designed to be model-agnostic. Additionally, the system does not address real-time EEG processing, clinical validation, or deployment in medical environments.

1.7 Terminology

- **SSVEP:** Steady-State Visual Evoked Potential, frequency-locked EEG response to periodic stimulation.
- **ITI:** Inter-Trial Interval, rest period between trials; in CCD, flickering gratings persist during ITI.
- **BIDS/HED:** Data and event annotation standards supporting consistent, analysis-ready datasets.

1.8 Personal Development Plan

This project is used as a competency-building exercise in software and AI engineering. The personal development goals are to:

- strengthen the ability to design reproducible ML experimentation pipelines,
- improve EEG-domain engineering skills (event handling, epoching, spectral verification),
- practice writing maintainable software (layering, modular design, configuration management), and
- improve reporting skills by connecting method choices, software artifacts, and evaluation results in a single narrative.

Chapter 2

Literature Review and Related Work

This chapter reviews related work relevant to prestimulus EEG-based behavioral prediction and to the software workflow required to run such experiments reproducibly. It first summarizes competing tools/workflows and then reviews foundational resources and key methodological insights that informed this project.

2.1 Competitor Analysis

Several existing tools support EEG preprocessing and modeling, but they differ in how directly they support a constrained prestimulus prediction setting (CCD-ITI) and how well they support end-to-end reproducible experimentation.

- **MNE-Python / MNE-BIDS:** strong for EEG preprocessing and BIDS interoperability, but experiment tracking and UI-driven workflows are typically built by the user.
- **EEGLAB / Brainstorm:** mature GUI tools for analysis and visualization, but reproducible pipeline execution and code-centric experiment comparisons may require additional scripting.
- **General ML notebooks:** flexible and fast to prototype, but prone to fragmentation (hidden state, inconsistent preprocessing, unclear configurations) unless carefully standardized.

This project complements these tools by emphasizing a traceable workflow tailored to the CCD-ITI prestimulus window: consistent event parsing and trial alignment, standardized preprocessing and feature extraction, and repeatable evaluation for both RT regression and correctness classification.

2.2 Literature Review

2.2.1 Foundational Works Leveraged

The EEG Foundation Challenge (EEG2025) provides the Healthy Brain Network EEG (HBN-EEG) dataset as a large-scale, high-density, multi-task EEG resource. Importantly for reproducible research, the dataset is distributed in BIDS format with HED annotations, which standardize file organization and event labeling across recordings.¹

- **HBN-EEG FAIR implementation (2024):** Presents analysis-ready EEG with integrated behavioral events and HED annotations, enabling reproducible analyses.²
- **HBN transdiagnostic resource (2017):** Describes the HBN biobank’s large-scale and community-sampled design to support dimensional research.³
- **EEG + eye tracking paradigms (2017):** Provides tasks including steady-state and contrast decision tasks supporting developmental brain investigations.⁴

2.2.2 Key Insights and Adaptations from Recent Methodologies

Based on these resources, this project adopts the following practical principles:

- **Event-locked trial construction is critical:** prestimulus prediction requires careful alignment between events, EEG windows, and behavioral labels.
- **Verification before prediction:** steady-state characteristics should be validated (e.g., via SNR spectra) to ensure that extracted features correspond to measurable signal properties.

¹[1, 2].

²[2].

³[3].

⁴[4].

- **Target-specific evaluation:** regression and classification require different metrics and error analyses; reporting should be consistent across runs.
- **Reproducibility as a software requirement:** pipeline parameters, dataset filters, and split strategies should be explicit to avoid misleading comparisons.

2.3 Positioning of This Work

This project uses HBN-EEG to study a focused research question: whether prestimulus EEG contains predictive information about behavioral performance. We model two outcomes: reaction time (RT) as a regression target and trial correctness (accuracy) as a binary classification target.

We evaluate this question in the Contrast Change Detection (CCD) task, where periodic visual stimulation during the rest period between trials supports steady-state feature extraction. By restricting inputs to a fixed 2-second prestimulus window, the modeling setting is intentionally constrained and emphasizes signal validation (via SNR spectra) and reproducible evaluation.

2.3.1 CCD Experimental Design

The CCD paradigm used in this study follows a structured visual stimulation and response protocol.

Stimuli and visual setup: Subjects view a continuously flickering annular stimulus (1° – 6° visual angle) composed of two overlaid gratings:

- Left-tilted grating (-45°): flickers at 20 Hz
- Right-tilted grating ($+45^\circ$): flickers at 25 Hz

At baseline, both gratings are presented at 50% contrast.

Target events: On each trial, one grating increases linearly to 100% contrast while the other simultaneously decreases to 0%. The contrast-change period lasts 1.6 s, followed by a 0.8 s return-to-baseline period. Targets are presented with an inter-target interval of 2.8 s, 4.4 s, or 6 s.

Participants respond using left-hand click for left-tilted contrast increase and right-hand click for right-tilted contrast increase.

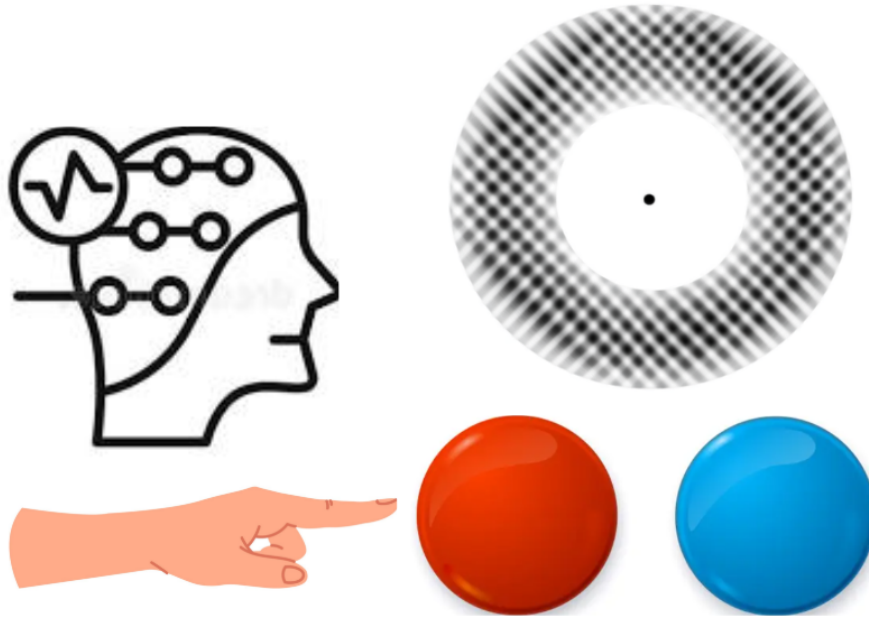


Figure 2.1: CCD experimental setup with fixation-centered visual stimulation and left/right response mapping.

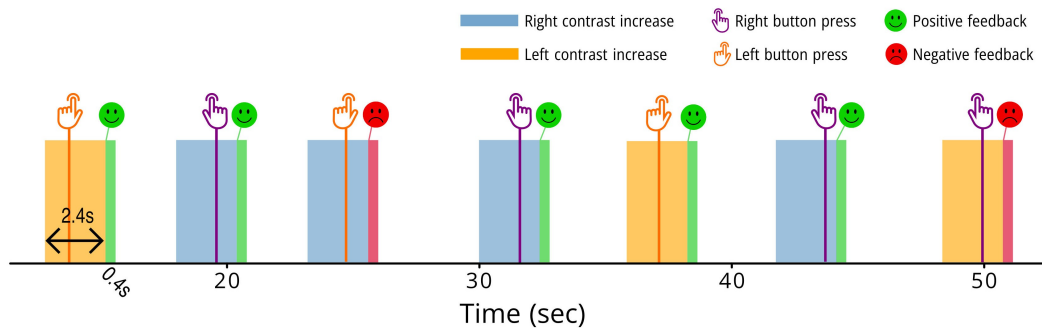


Figure 2.2: Example CCD trial sequence showing contrast-change events, button responses, and feedback.

Chapter 3

Requirement Analysis

This chapter defines the requirements for the *EEG Workbench* software used in this project. The system is implemented as a notebook-first, Python-based tool (`eegkit`) that orchestrates loading HBN-EEG (BIDS-like releases), preprocessing, CCD trial/ITI epoching, dataset building, model training, and export of plots/tables/models. The focus is not only to “run experiments”, but to make every run reproducible and auditable through cached pipeline artifacts and on-disk job specifications.

3.1 Stakeholder Analysis

The primary stakeholders of the proposed EEG analysis platform are researchers and students working in fields related to neuroscience, brain-computer interfaces (BCI), and EEG data analysis.

Researchers use the platform to conduct EEG data analysis, including signal inspection, preprocessing, and machine learning experimentation. Their work often involves multi-subject datasets and requires efficient tools for both individual and cohort-level analysis. Reproducibility and transparency are also critical, as experimental results must be consistent and verifiable.

Students use the platform for learning, experimentation, and project development. They often face challenges when working with complex EEG processing pipelines and scripting-based environments. The platform supports their needs by providing an interactive and structured interface that reduces technical barriers while still enabling advanced analysis capabilities.

In addition to primary stakeholders, the system also serves secondary stakeholders, such as machine learning practitioners or developers working with EEG data. These users may utilize the platform to prototype models, test data processing pipelines, or integrate new algorithms into the workflow. While they are not the main target users, the platform provides sufficient flexibility to support their experimentation needs.

Overall, the system is designed to support both experienced users and learners by simplifying EEG data workflows while maintaining the flexibility required for research and development.

3.2 Users and Interaction Modes

The workbench supports two interaction modes that correspond directly to the UI input schemas:

- **Single subject mode:** the user selects a subject, task name, and (optional) run identifier, then runs actions such as signal inspection plots, epoch extraction, and dataset/model generation.
- **Cohort filter mode (group):** the user selects a task and filters subjects by metadata (e.g., sex, age ranges) and task-derived CCD performance metrics (accuracy/response time). The user can optionally execute an action per subject to obtain comparable outputs.

3.3 User Stories

The following user stories describe the expected interactions between users and the system. These scenarios capture key functionalities required to support EEG data analysis and machine learning workflows.

- **US1 (Configure and load):** As a user, I want to point the tool to the HBN-EEG data directory and discover available subjects/tasks so that I can start analysis without manual path edits.

- **US2 (Inspect quality):** As a researcher, I want to preview raw/filtered EEG and view event tables so that I can verify signal quality and event integrity.
- **US3 (Reproducible preprocessing):** As a researcher, I want preprocessing/epoch parameters stored explicitly (band, notch, resample rate, channels, t_{min}/t_{max} , cleaning thresholds) so that results are comparable across runs.
- **US4 (Correct label alignment):** As a researcher, I want CCD trial outcomes to be aligned deterministically with trial anchors so that labels are not silently shifted.
- **US5 (Build datasets):** As a researcher, I want to build and save ML feature datasets and DL tensor datasets so that training can be repeated without re-deriving epochs each time.
- **US6 (Train/test and export):** As a researcher, I want model training/testing to output metrics and saved model artifacts so that I can compare approaches and report evidence.
- **US7 (Background execution):** As a user, I want long-running actions to run as background jobs with logs and saved outputs so that the notebook remains responsive.

3.4 Use Case Overview

The primary actor is the *Researcher*. The main use cases are: (1) configure and discover data, (2) inspect signals/events, (3) construct epochs and datasets, (4) train/test models, and (5) export and reproduce artifacts.

3.5 Use Case Model

3.5.1 Use Case 0: Configure Data Root

Goal: Configure the workbench to locate the HBN-EEG dataset on disk.

Main flow:

1. Set an environment variable (e.g., DATA_DIR / EEG_DATA_DIR) pointing to the dataset root.
2. Initialize the subject index, which discovers release folders and subject directories.
3. List available subjects/tasks to confirm correct data discovery.

3.5.2 Use Case 1: Inspect and Verify a Task Recording

Goal: Inspect a single subject/task/run and validate that events and signals look plausible.

Main flow:

1. Select *Single subject* input (subject, task, run).
2. Apply filtering/cleaning parameters (e.g., band-pass, notch, resample, channel selection).
3. View diagnostic plots (time domain, PSD/evoked/topomaps) and tables (events/channels/metadata).
4. Confirm that event markers needed for CCD preprocessing exist and are ordered correctly.

3.5.3 Use Case 2: Build CCD Epochs and Labels (Prestimulus/ITI)

Goal: Construct CCD trial-aligned epochs with deterministic outcome labels.

Main flow:

1. Select an epoch window (t_{min} , t_{max}) describing the prestimulus/ITI segment.
2. Create trial anchors from raw annotations (e.g., contrastTrial_start).

3. Derive trial outcome labels (hit/wrong/miss) from the parsed events table and strictly validate alignment to trial anchors.
4. Drop out-of-bounds epochs and interpolate bad channels when needed.
5. Cache derived epochs and labels so repeated analysis uses consistent data.

The ITI timeline in Figure 3.1 is used directly to define prestimulus window picking before target onset.

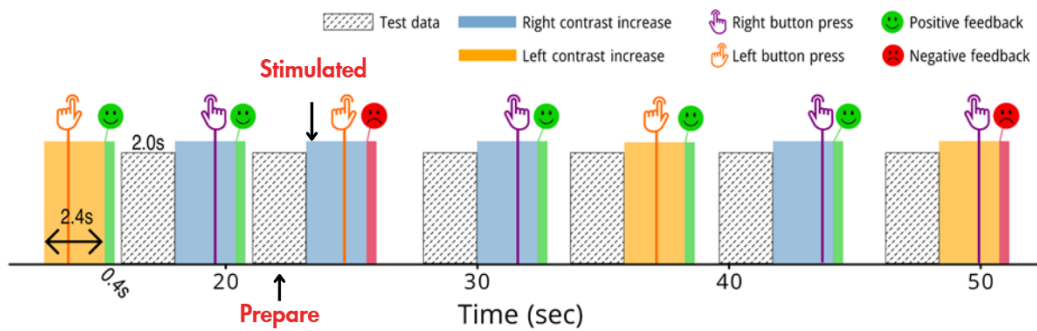


Figure 3.1: CCD trial timeline used to define ITI-window selection before target onset, with contrast-change events, button responses, and feedback markers.

3.5.4 Use Case 3: Build Training Datasets

Goal: Build datasets that can be reused by training runs.

Main flow:

1. Choose dataset type:
 - **ML feature dataset:** extract per-epoch feature vectors from selected channels.
 - **DL epoch tensor dataset:** export epoch tensors shaped for neural networks.
2. Persist arrays x , y , and $group$ (subject identifier) to the job output directory.

3. Save a dataset summary (shapes and class/group counts) for reporting and debugging.

3.5.5 Use Case 4: Train/Test Models and Export Artifacts

Goal: Train/test models and export both metrics and model artifacts.

Main flow:

1. Select a previously saved dataset (discovered under the `jobs/` directory).
2. Choose model type and hyperparameters.
3. Split data using group-aware strategies when possible (to avoid leakage across subjects).
4. Train and evaluate, then save: (i) model artifact, (ii) evaluation metrics JSON, and (iii) optional plots.

3.6 User Interface Design

The platform adopts a research-oriented interface design that aligns with the typical EEG analysis workflow. The interface is structured using a mode-action approach, where functionality is organized into context-specific operational modes.

The system provides four primary modes:

- **Plot Mode:** Enables detailed inspection of EEG signals for a single subject. Users can visualize raw or preprocessed signals to assess quality and identify patterns.
- **Grid Plot Mode:** Supports comparative visualization across multiple conditions or subjects. This mode facilitates identification of trends and differences at the cohort level.
- **Data Mode:** Provides structured access to dataset components, including metadata, event information, and preprocessing configurations. It serves as the primary interface for data preparation.

- **AI Mode:** Enables machine learning experimentation, including dataset construction, model training, and evaluation. Users can configure models and review performance metrics within this mode.

In addition to mode-based interaction, the system supports flexible subject selection through two approaches:

- **Single Subject Selection:** Users select an individual participant and task, allowing focused inspection and analysis.
- **Cohort Filtering (Meta Filter):** Users define filters based on attributes such as age range, sex, behavioral scores, or task performance. This enables large-scale exploratory analysis without manual subject selection.

Each mode exposes only the actions relevant to its context, reducing interface complexity and guiding users through the analysis workflow. Additionally, all user actions are logged to support transparency and reproducibility.

Input Type:

Single subject Meta filter (group)

subject: sub-NDARAC904DMU

task: DespicableMe

Mode:

Plot Grid Plot Data AI

Produces one concise figure for the current EEG selection.

Action:

Sensor Layout Time Domain Plot Frequency Domain Epoch Plot Evoked Plot Evoked Topo Plot Evoked Plot Joint

Evoked per Condition SNR Spectrum

☐ Show all groups

Filtering & Cleaning

l_freq:	4	h_freq:	30
notch:	60.0	resample_fs:	100
uv_min:	-100.0	uv_max:	100.0
clean_flatline_sec:	5.0	clean_hf_noise_sd_max:	4.0
clean_corr_min:	0.8	clean_asr_max_std:	20.0
clean_power_min_sd:	-100.0	clean_power_max_sd:	7.0
clean_max_outbound_pct:	25.0	clean_window_sec:	0.5
channels:	60-76,81-83,88,89		
combine_channels:	☐	showbad:	☐
clean_asr_remove_only:	☐		

Run on Tmux

Run Inline

Figure 3.2: Initial Notebook UI Design

Screenshots and detailed interaction flows are shown in Chapter 7.

3.7 Target and Development System

3.7.1 Data Handling and Input Capabilities

- Load HBN-EEG from a local directory containing release folders (e.g., `cmi_bids_R*`) with subject directories `sub-*` and EEG files (`*_eeg.set`).
- Parse per-subject task events (`*_events.tsv`) and metadata needed for CCD behavioral targets.
- Support single-subject selection and cohort filtering using participant metadata and CCD-derived performance fields.
- Cache pipeline artifacts (filtered raw, epochs, evoked) on disk with a pipeline-version stamp to prevent accidental reuse after logic changes.

3.7.2 Core Processing and Modeling Requirements

- Consistent preprocessing with explicit parameters: band-pass, notch, resampling, channel selection, amplitude bounds, and cleaning thresholds.
- Deterministic epoch extraction for CCD trial-aligned windows (pre-stimulus/ITI), including label alignment validation.
- Dataset building for both ML feature vectors and DL epoch tensors.
- Training/testing pipelines with repeatable split strategies; group-aware splitting should be used when sufficient subjects exist.

3.7.3 User Interface and Visualization Needs

- Plots for signal inspection (time domain previews, PSD/evoked/topographic views, and task-specific verification such as SNR where applicable).

- Tables for inspection (events, channels/electrodes, metadata, and epoch summaries).
- Job outputs saved to disk in a consistent directory structure so the report can reference stable file artifacts.

3.7.4 Non-Functional Requirements

- **Reproducibility:** parameter DTOs (filter/epoch/train) and random seeds are saved in job specs; cached artifacts include a pipeline-version stamp.
- **Traceability:** every exported artifact (plots, datasets, models) can be traced back to a job directory containing a `spec.json` and logs.
- **Robustness:** the pipeline must fail fast on label/epoch alignment mismatches rather than silently producing mis-labeled training data.
- **Maintainability:** modular components (participant discovery, loader, processor, services, UI) with registry-driven action discovery.

3.8 Functional Requirements

Table 3.1 summarizes the functional requirements (FR) derived from the user stories and the implemented workbench design.

ID	Requirement
FR1	The system shall accept a user-configured dataset root directory and discover available releases, subjects, tasks, and runs.
FR2	The system shall provide two input schemas: (i) single subject selection, and (ii) cohort filtering by participant metadata and CCD-derived performance ranges.
FR3	The system shall load raw EEG and event tables for a selected task and expose basic inspection views (plots and tables).
FR4	The system shall apply preprocessing using explicit parameters (band-pass, notch, resample rate, channel selection, amplitude bounds, and cleaning thresholds).
FR5	The system shall build CCD trial-aligned epochs for a user-specified window (t_{min} , t_{max}) and shall validate label alignment deterministically.
FR6	The system shall cache intermediate artifacts (filtered raw, epochs, evoked) keyed by task/run and parameter hashes, and invalidate caches via a pipeline-version stamp.
FR7	The system shall build and export ML feature datasets as arrays x , y , group and store a dataset summary for inspection.
FR8	The system shall build and export DL epoch tensor datasets (e.g., $N \times 1 \times C \times T$) with aligned labels and group identifiers.
FR9	The system shall train/test models on saved datasets and export evaluation metrics and the trained model artifact with metadata.
FR10	The system shall support both inline execution (notebook output) and background execution as a job with persisted specification, logs, and outputs.

Table 3.1: Functional requirements summary for the EEG Workbench.

3.9 Non-Functional Requirements

ID	Requirement
NFR1	Reproducibility: A run shall be reproducible from a saved job specification (input schema + params DTOs + seed) without manual notebook edits.
NFR2	Traceability: Outputs shall be saved in a stable directory structure with logs so figures/tables in the report can be traced to a specific run.
NFR3	Data integrity: The system shall raise an error on detected trial/label alignment mismatches, and shall avoid silently producing shifted labels.
NFR4	Performance: The system should use caching and cohort batching to reduce redundant processing when iterating on parameters.
NFR5	Usability: The UI shall expose sensible defaults for key parameters (e.g., filter band, channel list) and group actions into understandable modes.
NFR6	Maintainability/Extensibility: New tasks (preprocessors), plots, and AI actions shall be addable through registration/DTO-driven mechanisms without rewriting the UI layer.

Table 3.2: Non-functional requirements summary.

3.10 Extensibility (Easy to Extend)

An explicit goal of the EEG Workbench is that extending the system requires adding *new modules* rather than modifying existing UI logic. This is achieved by three design choices:

- **Registry-driven actions (plug-in style):** plotting, ML, and DL capabilities are exposed as action registries where each new action registers a *name*, a *parameter DTO class*, and a *handler function*. The UI reads the controller’s spec and automatically lists the new action.

- **DTO-driven parameters:** UI parameter panels are generated from dataclass DTOs (e.g., filter/epoch/train DTOs). Adding a new parameter set does not require hand-written widget layouts.
- **Task-specific preprocessors:** each dataset/task can provide its own preprocessing recipe via a lightweight registration mechanism (e.g., `register_preprocessor(task_name)`), so adding a new task does not require rewriting the pipeline core.

Table 3.3 summarizes typical extension scenarios and the minimal change surface expected.

Change case	What to add	Why it is low effort
EC1: Add a new plot	A new plot function + a params DTO (optional) registered into the plot action registry.	The UI reads the updated spec and auto-generates parameter widgets from the DTO; no UI refactor is required.
EC2: Add a new ML experiment	A new ML action registered via a decorator (name + DTO + handler) that consumes saved datasets.	Training/testing actions share common dataset discovery and job output conventions, so new actions reuse the same execution and export path.
EC3: Add a new DL model training	A new DL action with its own training DTO (batch size, epochs, learning rate, etc.) and handler.	DL actions are isolated behind the DL service; registering the action makes it available immediately in the same UI mode.
EC4: Support a new HBN task	A new task-specific preprocessor registered under a task name, including event parsing and epoch definitions.	The pipeline core calls a registered preprocessor per task; adding a new task does not require changing unrelated tasks or UI components.

Table 3.3: Extensibility change cases showing how new capabilities can be added with minimal modification to existing code.

3.11 Traceability (Stories to Requirements)

User story	Mapped requirements
US1	FR1, FR2
US2	FR3, FR4
US3	FR4, FR6, NFR1
US4	FR5, NFR3
US5	FR7, FR8, FR6
US6	FR9, NFR2
US7	FR10, NFR2

Table 3.4: Traceability matrix from user stories to requirements.

Chapter 4

Software Architecture Design

This chapter presents the software architecture that supports reproducible EEG experimentation in this project. It describes a layered architecture, the main execution flow, the AI processing pipeline, and the key data schemas used to make runs traceable and repeatable.

4.1 Software Architecture

The system is designed using a **layered client-server architecture**, consisting of a React-based frontend and a FastAPI backend. The backend is implemented as a modular monolith, where functionalities are organized into domain-specific modules such as API handling, data processing, visualization, and machine learning.

At its core, the system follows a **pipeline-oriented architecture**, where EEG data is processed through sequential stages (e.g., preprocessing, epoch extraction, and feature generation). This structure enables reproducibility and flexible recomputation when parameters change.

4.2 Domain Model

The **domain model** of the system represents the key entities and their relationships involved in EEG data analysis and visualization. The platform is designed around a research-oriented workflow, supporting both **single-subject analysis** and **cohort-based analysis**.

Core Entities

The primary domain entities are defined as follows:

- **Subject:** Represents an individual participant in the dataset. Each subject contains multiple experimental tasks and associated EEG recordings.
- **Task:** Represents a specific experimental condition or recording session associated with a subject. Each task serves as the entry point for EEG data processing.
- **EEG Raw Data:** The original recorded EEG signals loaded from disk. This data is the starting point of the processing pipeline.
- **Processed Signals:** Intermediate representations of EEG data, including:
 - **Filtered Signals:** Signals after applying frequency filtering and noise removal.
 - **Cleaned Signals:** Signals after artifact removal and preprocessing.
- **Epochs:** Segmented EEG data derived from raw signals based on event markers. Epochs enable time-locked analysis of brain responses.
- **Evoked Responses:** Averaged EEG signals across epochs, representing stimulus-locked neural activity.
- **Cohort:** A dynamically defined group of subjects selected using filtering criteria (e.g., task type, demographic attributes). Cohorts enable batch analysis across multiple subjects.
- **Model:** Represents machine learning models used for training and evaluation. Models consume processed EEG features derived from epochs or evoked data.
- **Visualization Output:** The final output of the system, which can be either:
 - **Graphical plots** (e.g., time-domain, frequency-domain, topographic maps).
 - **Tabular data** (e.g., EEG metadata, feature tables, training summaries).

Relationships Between Entities The relationships between the entities are structured as follows:

- A **Subject** contains multiple **Tasks**.
- A **Task** produces **EEG Raw Data**.
- **EEG Raw Data** is transformed into **Processed Signals** and **Epochs**.
- **Epochs** are further transformed into **Evoked Responses** and **Features** for **Model** training.
- A **Cohort** is a collection of multiple **Subjects**, each processed independently.
- **Visualization Output** is generated from any stage of the pipeline depending on user-selected actions.

Processing Flow

The domain model follows a sequential processing pipeline:

Subject → **Task** → **Raw Data** → **Signal Cleaning/Filtering** → **Epoching** → **Evoked/Features** → **Visualization** or **Model Training**

4.3 Design Patterns

To ensure a modular and extensible architecture capable of handling high-throughput data, the following design patterns were implemented:

- **Pipeline Pattern:** EEG processing workflow
- **Facade Pattern:** API endpoints
- **Strategy Pattern:** parameter-driven behavior
- **Factory Pattern:** model instantiation
- **Registry Pattern:** task preprocessor selection
- **Observer Pattern:** WebSocket progress updates

- **Repository Pattern:** structured EEG cache
- **Composite Pattern:** Pipeline Session is a combination of schemas

4.4 Design Class Diagram

The system adopts a modular and layered design, where responsibilities are separated across execution, processing, visualization, and infrastructure components.

Core Execution Components

- **TaskExecutor:** Handles the execution of the EEG processing pipeline for a single **Subject–Task** pair. It is responsible for:
 - Loading raw EEG data.
 - Applying preprocessing steps (filtering, cleaning).
 - Generating **epochs** and **evoked responses**.
 - Dispatching results to visualization or training modules.
- **CohortExecutor:** Manages batch processing across multiple subjects. It maintains a collection of **TaskExecutor** instances and executes them iteratively. Additional validation and aggregation logic are applied at this level.

Processing Pipeline Components

- **SignalCleaner:** Performs artifact removal and preprocessing on raw EEG signals.
- **SignalSpatial:** Handles spatial transformations and channel-level operations.
- **TaskLoader / TaskResolver / TaskProcessor:** These components collaboratively define and execute processing steps:
 - **TaskLoader:** Loads task-specific configurations and data.
 - **TaskResolver:** Determines the required processing steps based on user input.

- **TaskProcessor**: Executes the processing pipeline.
- **TrainerDataBuilder**: Prepares processed EEG data for machine learning tasks.

Visualization Components The visualization system is divided into three categories:

- **Plot Modules (plot/)**: Generate single visualizations such as time-domain plots, frequency plots, and topographic maps.
- **Grid Plot Modules (grid_plot/)**: Generate multi-condition visualizations for comparative analysis.
- **Data and AI Views (data/, ai/)**: Produce tabular outputs such as EEG summaries, metadata, and model training results.

Supporting components include:

- **PlotMerger**: Combines multiple figures into a unified visualization.
- **PlotFinalizer**: Applies consistent styling and formatting across all plots.

Schema Layer

- **Parameter Schemas**: Define input parameters for filtering, visualization, and training. These schemas serve both:
 - **Request validation** (FastAPI layer).
 - **UI rendering** (dynamic parameter generation).
- **UI Schemas**: Define session state, task configuration, and view structure.

Infrastructure Components

- **CacheManager**: Stores intermediate processing results (e.g., cleaned signals, filtered signals, epochs) using a hierarchical structure: **Subject** → **Task** → **Processing Stage** → **Parameter Hash**.
- **ParticipantsLoader**: Loads subject metadata and dataset structure.

- **ProgressLogger & WebSocketManager**: Provide real-time feedback to the frontend during long-running operations.

Interaction Flow

1. The user selects a mode (single or cohort) and defines parameters via **schemas**.
2. The system initializes either a **TaskExecutor** or **CohortExecutor**.
3. The execution layer processes EEG data through the pipeline.
4. Intermediate results are **cached** to optimize repeated computations.
5. The output is passed to **visualization modules** (plot/grid/data/ai).
6. Final results are returned to the user interface.

Design Characteristics

- **Modularity**: Each component has a clearly defined responsibility.
- **Reusability**: Processing steps are reused across visualization and training workflows.
- **Scalability**: **CohortExecutor** enables batch processing across multiple subjects.
- **Extensibility**: New plots, filters, and models can be added without modifying core logic.

The **Visualization module** (app/plots) contains submodules for different types of outputs:

The **AI Model module** (app/ai_models) defines machine learning models used within the platform. These models can be trained and evaluated using processed datasets.

Additionally, the system maintains a **job/output storage structure** (jobs/ai/train/...) that stores training results, logs, and model artifacts, along with the cache for each pipeline step in **eegcache** folder (such as prefiltered, cleaned, epochs, etc.) While not a full asynchronous job system, this structure supports persistence and traceability of preprocessing and model training runs.

This modular design ensures separation of concerns, making the system maintainable, extensible, and aligned with the research workflow.

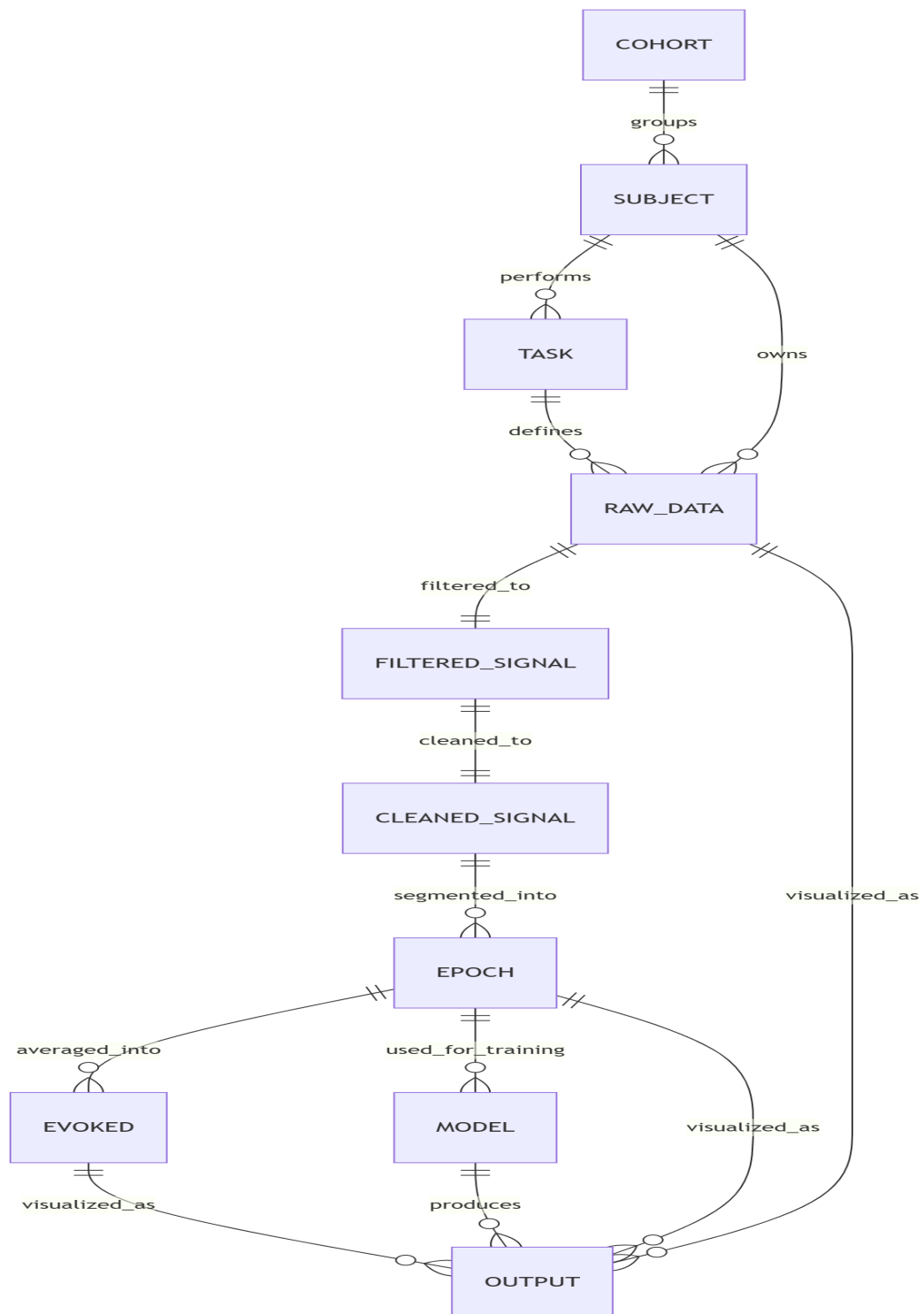


Figure 4.1: Domain Model Diagram

4.5 Sequence Diagram

4.5.1 Experiment Execution Flow

Scenario: Run an experiment

1. The user selects a subject and task from the Web UI.
2. The Web UI sends an experiment-run request with parameters to the Backend API.
3. The Backend API requests the Pipeline to load EEG and event meta-data, then returns a quick validation summary to the UI.
4. The user confirms the run after reviewing the validation summary.
5. The Backend API triggers preprocessing and epoch construction in the Pipeline.
6. The Pipeline builds features and forwards data to the AI Trainer for training and validation.
7. The trainer returns metrics and predictions, artifacts are persisted in the Result Store, and the UI displays the run summary (plots and model scores).

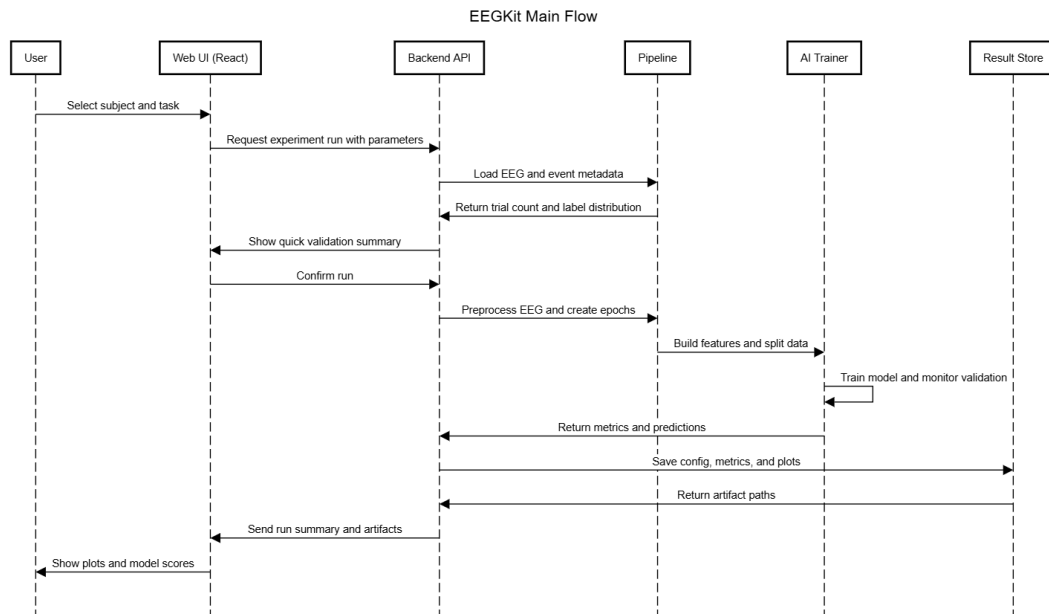


Figure 4.2: EEGKit main interaction flow (Web UI → Backend API → Pipeline → AI Trainer/Result Store)

4.6 Pipeline

4.6.1 AI Processing Pipeline

The AI processing pipeline is designed to enforce traceability. Each stage consumes explicit inputs and produces versioned outputs:

1. **Ingestion:** load CCD runs and parse events/metadata.¹
2. **Trial construction:** extract fixed-length ITI windows and attach labels.
3. **Preprocessing:** filter, normalize, mitigate artifacts, and create epochs.
4. **Verification:** compute SNR spectra and related plots for steady-state validation.
5. **Features:** compute ITI-derived features (e.g., spectral power at stimulation frequencies/harmonics).

¹[2].

6. **Modeling:** train RT regressors and correctness classifiers.
7. **Evaluation:** compute metrics and export artifacts for reporting.

Chapter 5

AI Component Design

This chapter describes how the AI component is integrated into the overall system workflow, from dataset construction and signal verification to model training and evaluation. It follows an AI-canvas-style requirement analysis, documents model development and evaluation, and describes how the user experience and deployment support reproducible experiments.

5.1 Business Context and AI Integration

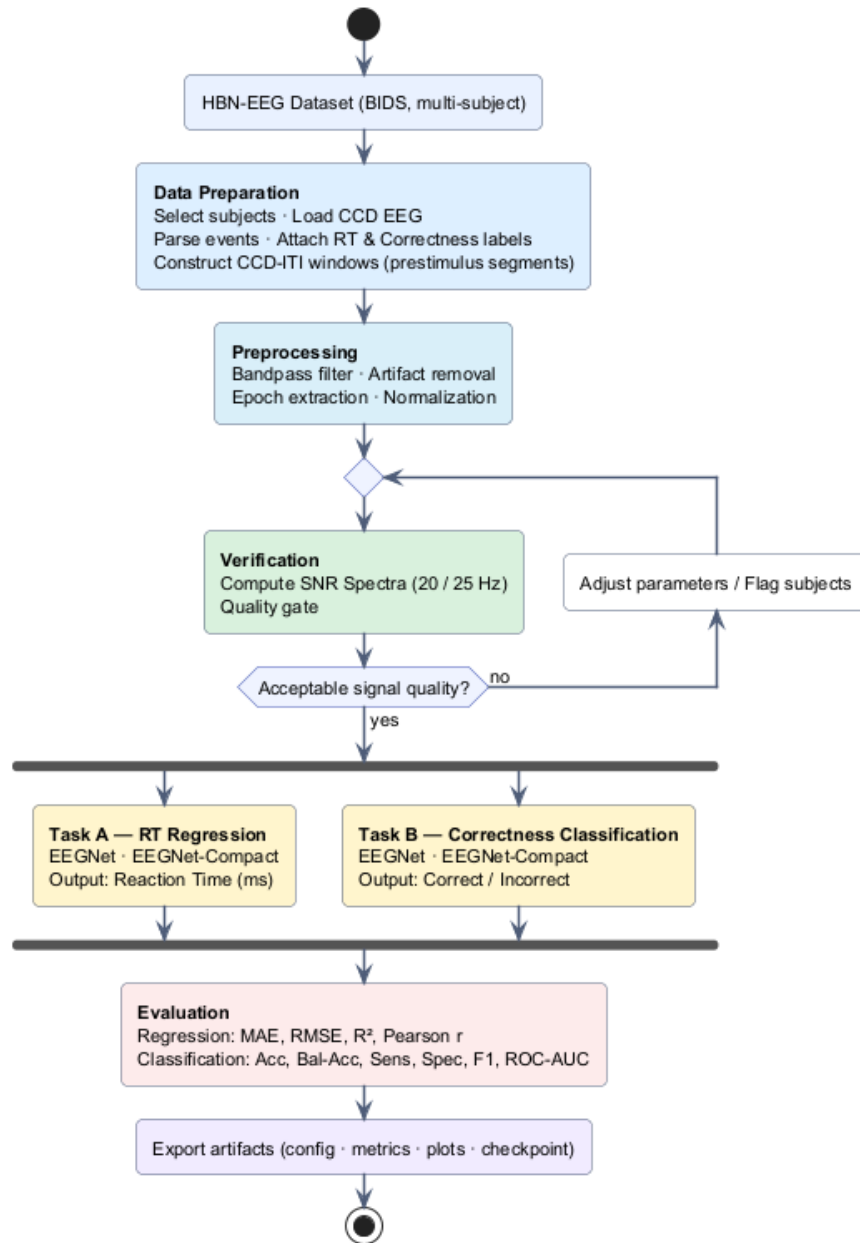


Figure 5.1: Workflow for prestimulus EEG-based behavioral prediction from CCD-ITI segments.

Diagram Explanation: In this system, artificial intelligence (AI) is incorporated as an experimentation tool to support EEG-based analysis and model development. Rather than functioning as a standalone predictive system, the AI component is designed to facilitate the process of preparing data, training models, and evaluating results within an integrated workflow.

The platform enables users to work with preprocessed EEG data and apply machine learning models to explore relationships between neural signals and behavioral outcomes. In particular, the system supports the evaluation of models that predict behavioral responses, such as reaction time and task correctness, based on EEG signals.

The inclusion of AI functionality allows users to transition seamlessly from data exploration to model experimentation, reducing the need to switch between multiple tools or environments.

The workflow proceeds as: signal inspection → dataset verification → model training → evaluation.

This ensures that model results are directly connected to verified signal characteristics rather than blindly training on raw data.

AI Suitability for the Problem

EEG analysis involves high-dimensional temporal signals with complex spatial relationships across electrodes. Manual feature engineering is difficult because:

- patterns vary across subjects
- noise and artifacts are common
- signal distributions change across recording sessions

Machine learning models can learn robust mappings from EEG-derived features to behavioral targets more effectively than fixed heuristics.

Problem Scale and Complexity

EEG data presents three major challenges:

- Large — continuous multi-channel time series
- Complex — spatial-temporal correlations
- Non-stationary — signals differ between subjects and sessions

Because of this variability, deterministic rule-based systems are unsuitable.

Acceptable Performance Criterion

Because this is a research system, predictions are expected to be informative rather than perfect.

The system is intended for research experimentation rather than safety-critical decision making. Model predictions are used as analytical indicators, not medical diagnoses. Therefore approximate predictions with measurable error are acceptable.

5.2 Goal Hierarchy

The goals of the AI component are structured across multiple levels to reflect organizational intent, system functionality, user needs, and model-specific objectives.

Organizational Goals:

- Support efficient and reproducible EEG data analysis
- Facilitate research in EEG-based behavioral prediction
- Provide a unified platform for data exploration and machine learning experimentation

System Goals:

- Integrate data preprocessing, visualization, and machine learning into a single workflow
- Enable reusable and parameter-driven data processing
- Support training and evaluation of machine learning models
- Reduce redundant computation through caching mechanisms

User Goals:

- Explore EEG signals interactively
- Configure preprocessing and analysis parameters

- Build datasets for machine learning
- Train and evaluate models efficiently
- Compare results across different configurations or subjects

AI Model Goals:

- Learn mappings between EEG-derived features and behavioral outcomes
- Predict reaction time (regression task)
- Predict trial correctness (classification task)
- Generalize across multiple subjects and varying EEG conditions

Success Metrics:

- reduced prediction error (e.g., MAE/MSE) for RT
- improved correctness prediction (accuracy, F1-score, precision, recall)
- consistent performance on unseen subjects
- reduced experiment setup time
- reproducible experiment workflow

5.3 Task Requirements Analysis Using AI Canvas

5.3.1 AI Task Requirements

- **Requirements (REQ):** Predict behavioral outcomes from pre-stimulus CCD-ITI EEG segments in HBN-EEG.
- **Specifications (SPEC):** Evaluate models that map CCD-ITI features to RT (regression) and trial correctness (classification).
- **Environment (ENV):** Evaluated on multi-subject EEG datasets with different recording conditions. (e.g., noise, subject variation).

5.3.2 AI Canvas Summary

- **Input:** Preprocessed EEG segments
- **Output:** Predicted RT (regression) and predicted correctness (classification)
- **Success Criteria:** Lower RT prediction error and higher correctness classification performance with stable generalization to unseen subjects.

5.3.3 Innovation

The system integrates visualization-driven dataset validation with model training inside a single interactive interface, reducing mismatch between inspected signals and training data.

5.4 AI Model Development and Evaluation

5.4.1 Model Development and Training Strategy

The modeling workflow is designed to be repeatable and target-aware:

- **Inputs:** preprocessed CCD-ITI epochs and ITI-derived feature tables.
- **Targets:** RT (regression) and correctness (binary classification).
- **Baseline-first approach:** start with simple, interpretable models before considering more complex architectures.
- **Split strategy:** prefer subject-independent splits to evaluate generalization and reduce overly optimistic estimates.

5.4.2 Testing and Validation Approach

Validation focuses on preventing misleading results caused by data issues or leakage:

- sanity checks for trial counts, label distribution, and missing channels,

- verification plots (e.g., SNR spectra) to confirm steady-state characteristics,
- consistent preprocessing parameters and fixed random seeds,
- run logging of dataset filters, split definitions, and model configurations.

5.4.3 Performance Results and Justification

Recent experimental results were analyzed for both targets (RT regression and correctness classification). The evaluation intentionally avoids selecting a single “best model” because each configuration fails in a different way.

The research process required multiple difficult iterations before obtaining stable trainable pipelines. We repeatedly revised trial alignment, preprocessing consistency, split definitions, and logging before results became comparable across runs. Even after that stabilization, predictive quality remained limited.

Model results are reported using target-appropriate metrics:

- **RT regression:** MAE, RMSE, R^2 , and Pearson correlation.
- **Correctness classification:** accuracy and balanced accuracy, with sensitivity/specificity and confusion-matrix behavior.
- **Training dynamics:** training loss and validation loss across epochs.

Observed patterns across the evaluated configurations are:

- **Regression stagnation:** MAE remains narrowly clustered around 0.287–0.288 and R^2 remains near zero (about 0.003–0.008), indicating weak explanatory power.
- **Mean-prediction tendency:** predicted RT mean is close to true RT mean, suggesting partial collapse toward central tendency instead of learning richer trial-level structure.
- **Imbalance-driven classification failure:** one high-accuracy configuration reaches about 0.729 accuracy but only about 0.501 balanced

accuracy, with very high sensitivity and very low specificity, indicating majority-class dominance.

- **Partial mitigation only:** class weighting, weighted sampling, and undersampling were introduced specifically to address class imbalance; they improved specificity somewhat, but none produced consistently strong balanced performance.

These outcomes show that class imbalance is an important contributor, but not the only bottleneck. The current AI component should therefore be treated as a reproducible experimental baseline rather than a finalized predictive solution.

5.5 User Experience Design with AI

The platform incorporates AI functionality through a dedicated **AI Mode**, which allows users to perform machine learning-related operations within the same interface used for data exploration.

In this mode, users can:

- Configure datasets derived from preprocessed EEG data
- Execute model training procedures
- Evaluate model performance using standard metrics

The system presents outputs in a structured format, including tabular summaries of evaluation metrics and execution logs. This enables users to monitor model performance and understand the outcome of training processes.

The integration of AI functionality within the broader platform ensures a seamless workflow, where users can move from signal inspection to model experimentation without switching environments. This design supports iterative experimentation and reduces the complexity associated with traditional notebook-based workflows.

5.6 Deployment Strategy

The system is designed to support a hybrid deployment using a modern client-server architecture, balancing remote accessibility with necessary computational power.

The **frontend** is deployed using **Vercel**, which provides a scalable and efficient hosting environment for React-based applications. This enables users to access the platform through a web interface with minimal setup.

The **backend** is deployed locally using **Docker** containerization, with external access securely routed through **Ngrok**. This transition from cloud hosting was necessary because standard free-tier cloud environments

lack sufficient Random Access Memory (RAM) to accommodate the heavy computational and memory demands of the machine learning models.

This deployment approach ensures that the platform remains easily accessible remotely via the cloud-hosted frontend, while strategically bypassing memory constraints by leveraging local hardware for the backend. Furthermore, the use of Docker ensures environmental consistency and seamless portability for future infrastructure upgrades.

5.7 Proof of Concept (Service Demonstration)

The pipeline can be exercised end-to-end under a subject-independent evaluation setting. In this setting, models are trained on a group of subjects and tested on unseen individuals, which better reflects generalization and avoids overly optimistic estimates from within-subject testing.

The system supports the workflow:

visualization → dataset selection → training → inference → evaluation

Results can be accessed through the interface and (when enabled) through API endpoints.

Chapter 6

Software Development

This chapter documents how the software was developed to support the project's research workflow. It describes the development methodology, key architectural patterns used to keep experimentation maintainable, and the technology stack used to implement preprocessing, modeling, evaluation, and the supporting interface.

6.1 Software Development Methodology

The development of the proposed EEG analysis platform follows an Agile software development methodology. Agile emphasizes iterative development, flexibility, and continuous improvement, making it well-suited for research-oriented systems where requirements may evolve over time.

In the context of this project, Agile supports incremental implementation of system features, allowing the platform to be gradually refined based on testing and usability considerations. This approach is particularly appropriate for EEG data analysis systems, where workflows often require adjustment as new insights are gained during development.

The development process followed four repeating steps:

- M1. Define a measurable milestone:** e.g., extract CCD ITI prestimulus windows, compute features, train a regressor, or produce evaluation plots.
- M2. Implement and validate:** develop the smallest working version of the milestone, then validate with quick sanity checks (shape checks, trial counts, summary statistics, and train/validation splits).

M3. Evaluate and document: run a consistent evaluation protocol and record results, assumptions, and limitations.

M4. Refine: address issues found in evaluation (data leakage risks, unstable preprocessing, model underfitting/overfitting) before moving to the next milestone.

To keep the work aligned with the project scope, the model development emphasized:

- **Clear data boundaries** between subjects/sessions when creating splits.
- **Simple models first** (feature-based regressors) before attempting more complex architectures.
- **Reproducibility** through deterministic preprocessing parameters, fixed random seeds, and consistent evaluation scripts.

6.2 Technology Stack

The platform is implemented using a combination of modern web technologies and scientific computing libraries, chosen to support both user interaction and EEG data processing.

- **Frontend:** Developed using React with TypeScript, providing a responsive and structured user interface for interacting with the system.
- **Backend:** Implemented using FastAPI in Python, which handles API requests, data processing, and communication between system components.
- **EEG data processing:** Utilizes the MNE library, which provides tools for handling EEG data in standardized formats such as BIDS.
- **Artifact removal:** Employs the ASRpy library to support artifact removal through the Artifact Subspace Reconstruction method.
- **Data manipulation and computation:** Performed using NumPy and Pandas.
- **Data visualization:** Generates visual representations using Matplotlib.

- **Version control:** Git for change tracking and branching.

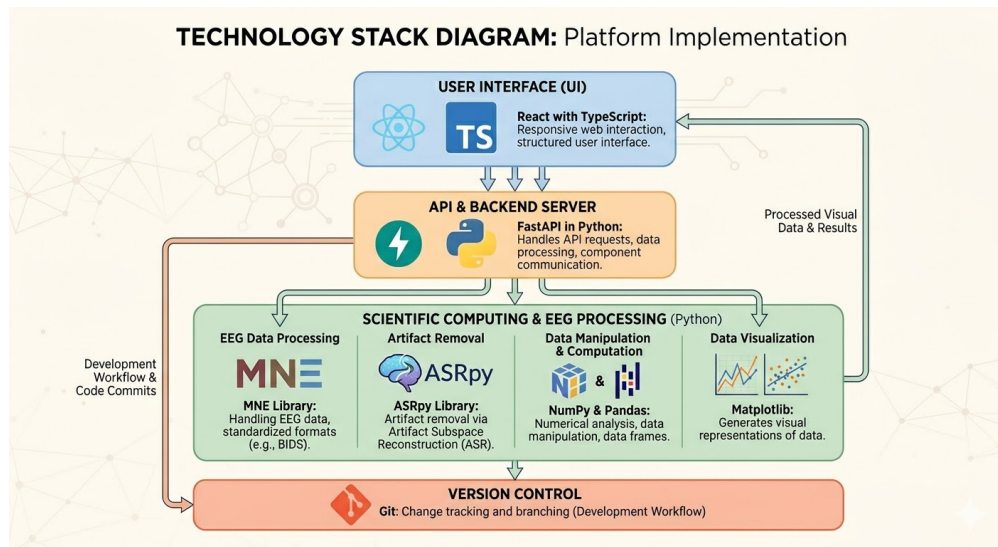


Figure 6.1: Tech Stack Diagram

6.3 Coding Standards

The codebase follows a small set of conventions to keep experiments easy to reproduce and results easy to interpret.

- **Modular structure:** preprocessing, feature extraction, training, and evaluation are separated into distinct modules to reduce coupling.
- **Configuration over hard-coding:** key parameters (window length, frequency bands, feature sets, split strategy, model hyperparameters) are stored in configuration objects/files.
- **Reproducible runs:** fixed random seeds, explicit train/validation/test split definitions, and logging of dataset filters and preprocessing parameters.
- **Consistent naming:** functions and variables are named after domain concepts (trial, epoch, prestimulus window, RT target, accuracy target) rather than implementation details.

- **Input validation:** early checks for missing channels, unexpected sampling rates, and mismatched trial labels to avoid silent failures.
- **Documentation:** concise docstrings for public functions, and short experiment notes describing what changed between runs.

For evaluation outputs, plots and metrics are produced by scripts that can be rerun end-to-end, ensuring that reported results can be regenerated from the same inputs.

6.4 Progress Tracking Report

Work was tracked by incremental milestones, with each milestone producing a tangible artifact (code, plots, tables, or document sections). Table 6.1 summarizes the main development stages.

Phase	Activities	Outputs
1: Setup & scoping	Repository setup, project scope definition (CCD ITI prestimulus), and report structure alignment	Buildable L ^A T _E X document, chapter plan
2: Data understanding	Explore HBN-EEG metadata/task structure; identify CCD ITI segments and behavioral targets (RT, accuracy)	Dataset summary notes, initial sanity plots
3: Preprocessing pipeline	Implement prestimulus window extraction and consistent preprocessing parameters; verify trial counts and split boundaries	Reusable preprocessing scripts, cached intermediate outputs
4: Model development	Train models for RT (regression) and correctness (classification); compare simple model families and feature sets	Evaluation metrics (MAE / MSE for RT; accuracy / F1 / precision / recall for correctness) and prediction vs. ground-truth plots
5: Evaluation & reporting	Define evaluation protocol, error analysis, and limitations; integrate key results into deliverables chapter	Evaluation tables/figures
6: UI documentation	Document the notebook-style interface used for exploration and experiment logging	UI screenshots and usage description

Table 6.1: Progress tracking summary for the prestimulus behavioral prediction project.

6.5 Final Iteration Status

At the final reporting stage, the software workflow is complete for reproducible experimentation (dataset building, preprocessing, training/evaluation execution, logging, and artifact export). However, model quality remains an open technical problem.

In practice, the team passed through many technical challenges before reaching a stable modeling stage. The main effort was not only training models, but making the pipeline reliable enough that results could be trusted and compared.

The remaining issues are:

- **Classification instability under class imbalance:** high headline accuracy can hide poor minority/negative-class behavior.
- **Regression underfitting:** RT models show near-zero R^2 across multiple loss functions.
- **Imbalance mitigation did not fully solve performance:** class weighting, weighted sampling, and undersampling reduced some bias effects but did not produce robust, generalizable performance.
- **Limited time for broader sweeps:** some planned ablations (broader hyperparameter and architecture studies) were deferred to future work.

Therefore, this final deliverable should be interpreted as a stable engineering baseline and validated experimentation platform, while predictive performance improvements are left as a focused next phase.

Chapter 7

Deliverable

This chapter summarizes the tangible outputs produced by the project and the test/evaluation protocol used to assess them. It focuses on what is delivered (software and artifacts) and how performance is measured for RT regression and correctness classification.

7.1 Software Solution

The proposed system is an interactive EEG analysis platform designed to support data exploration, preprocessing, visualization, and machine learning experimentation within a unified interface. The platform is developed to address inefficiencies in traditional notebook-based workflows by providing a structured and parameter-driven environment for EEG analysis.

For more, please refer to our **Github**:

[Link: <https://github.com/Nantawat6510545543/senior-project>] for the web platform development, and

[Link: <https://github.com/Nantawat6510545543/eeg2025-Vistec>] for the model training development.

7.1.1 System Overview

The platform follows a research-oriented workflow where users first inspect signals before executing machine learning models. To support this process, the interface uses a **mode-action** structure, and additionally provides flexible **subject selection and filtering** to accommodate both single-subject inspection and cohort-level analysis.

7.1.2 Input Configuration

Users can choose between two primary input types:

- **Single Subject:**
 - User selects an individual participant and task.
 - All mode actions are executed for the selected subject only.
 - This mode is ideal for detailed inspection or troubleshooting of individual EEG recordings.
- **Cohort Filter (Meta Filter):**
 - User defines filters across attributes: sex, age range, attention, internalizing/externalizing scores, CCD accuracy, response time.
 - Optional “Per Subject” toggle for applying filters individually or collectively.
 - This mode enables exploratory data analysis and large-scale EEG studies without manual participant selection.

In both cases, the selected input determines the scope of subsequent operations.

7.1.3 Mode-Based Interaction

The platform organizes functionality into four primary operational modes:

- **Plot Mode:** Provides detailed visualization of EEG signals. This mode supports both single-subject and cohort-based data, enabling focused signal inspection.
- **Grid Plot Mode:** Enables comparative visualization across multiple subjects, conditions, or cohorts using structured layouts.
- **Data Mode:** Presents structured tabular representations of EEG data and derived results, allowing users to inspect intermediate outputs.
- **AI Mode:** Supports machine learning experimentation, including dataset preparation, model training, and evaluation.

Each mode dynamically exposes relevant actions, allowing users to interact with the system in a structured and context-aware manner.

7.1.4 Action Selection

Each operational mode in the platform exposes a set of actions that correspond to specific functionalities. These actions are organized within their respective mode (Plot, Grid Plot, Data, AI), enabling users to interact with the system in a structured manner.

Plot Mode Actions

Provides detailed visualization of EEG signals. Available actions include:

- **Evoked** – Visualizes averaged evoked responses for individual conditions or overall.
- **Frequency** – Generates frequency-domain representations of EEG signals.
- **Sensors** – Displays the spatial arrangement and activity of EEG sensors.
- **SNR** – Computes and visualizes the signal-to-noise ratio across channels.
- **Time Domain** – Shows EEG signal waveforms in the time domain.
- **Epochs** – Visualizes segmented epochs of EEG data.
- **Evoked Joint** – Combines multiple evoked responses into a single visualization.
- **Evoked Per Condition** – Displays evoked responses separately for each experimental condition.
- **Evoked Topography** – Maps spatial topography of evoked responses across electrodes.

Grid Plot Mode Actions

Enables comparative visualization across multiple subjects or conditions:

- **Evoked Grid** – Arranges evoked responses for multiple subjects in a structured grid.
- **PSD Grid** – Displays power spectral density comparisons across subjects or conditions.
- **SNR Grid** – Shows signal-to-noise ratio comparisons for multiple subjects in a grid layout.

Data Mode Actions

Provides tabular inspection of EEG data and derived metrics:

- **EEG Table** – Presents raw or preprocessed EEG data in tabular form.
- **Epochs Table** – Displays segmented EEG epochs and associated metadata.
- **Metadata** – Shows participant and recording metadata in a structured table.

AI Mode Actions

Supports machine learning experimentation, including dataset preparation and model evaluation:

- **Build Dataset** – Prepares and visualizes the training dataset for machine learning models.
- **Model Summary** – Displays architecture and parameters of the selected AI model.
- **Show Train EEG** – Executes training routines on EEG data and presents training metrics in a structured table.

7.1.5 Execution and Logging

The platform executes user-selected operations in a synchronous manner. During execution, an execution **Log Panel** is displayed, providing real-time feedback on system operations such as data loading, preprocessing steps, and plot generation.

To improve efficiency, the system implements a **caching mechanism**. Intermediate results, including prefiltered signals, cleaned data,

and epochs, are stored based on subject and parameter configurations. When the same configuration is reused, the system retrieves cached results instead of recomputing them.

This approach reduces redundant computation and supports faster iteration during analysis.

The platform presents outputs directly within the interface. Visualization results are rendered as plots, while structured outputs are displayed as tables.

7.1.6 System Characteristics

The developed platform demonstrates the following characteristics:

- **Interactive:** Enables real-time configuration and execution of analysis tasks
- **Reproducible:** Maintains explicit parameter configurations and execution logs
- **Efficient:** Reduces redundant computation through caching mechanisms
- **Flexible:** Supports both single-subject and cohort-level analysis
- **Integrated:** Combines preprocessing, visualization, and machine learning in a unified system

Feedback Loop: inspect → adjust → train → evaluate → compare → refine

Researchers can iteratively adjust dataset configurations, rerun experiments, and retrain models without rewriting scripts or managing computing environments. This significantly shortens the experimental cycle compared to notebook-based pipelines.

7.2 Test Report

This section describes the testing approach used to evaluate the correctness and reliability of the proposed EEG analysis platform. The evaluation focuses on functional verification, reproducibility, and system behavior under typical usage scenarios.

7.2.1 Testing Approach

The system was tested using a functional testing approach, where each core component was evaluated independently and as part of the overall workflow. The testing process focuses on verifying that user actions produce correct outputs and that the system behaves consistently across repeated executions.

Testing was performed through manual interaction with the user interface, covering different operational modes and configurations. This approach ensures that the system supports the intended research workflow and that all features are accessible and usable.

7.2.2 Functional Testing

Functional testing was conducted to verify that each system component operates as expected under standard usage conditions. The primary objective of the development was to validate the “Expected usage scenario”—ensuring that the core research workflow from data loading to machine learning execution functions correctly for valid, expected inputs.

The testing process focused on verifying that user actions produce correct visual and tabular outputs, and that the system behaves consistently across repeated executions. To systematically evaluate this, a core suite of functional test cases was executed covering the platform’s primary operational modes. The specific scenarios and their outcomes are summarized in Table 7.1.

Test ID	Component	Scenario / Action	Expected Result	Status
TC-01	Input Con-fig	Load a standard, uncorrupted EEG dataset for a Single Subject.	System successfully parses the file, maps channels, and initializes the Plot Mode.	Pass
TC-02	Cohort Filter	Apply a valid filter configuration (e.g., specific subject limit).	System filters the dataset and loads only matching subjects into the active cohort.	Pass
TC-03	Preprocessing	Apply a 50Hz bandpass filter and execute the “Frequency” plot.	Plot generates successfully, reflecting the expected frequency attenuation.	Pass
TC-04	Grid Plot	Execute “Evoked Grid” for a multi-subject cohort with valid condition markers.	UI renders a structured grid layout comparing evoked responses across subjects.	Pass
TC-05	Data Mode	Access the “Epochs Table” after segmenting the EEG data.	Table populates with correctly structured rows representing the segmented epochs.	Pass
TC-06	Caching	Re-execute TC-03 without changing any parameters.	System bypasses filtering computation, retrieving the array from cache to render faster.	Pass
TC-07	AI Dataset	Execute “Build Dataset” on a valid preprocessed subject.	System structures the tensors correctly and displays the data summary in the UI.	Pass
TC-08	ML Execution	Run “Show Train EEG” using default regression parameters.	Training loop executes synchronously and outputs MAE, RMSE, and R^2 metrics.	Pass

Table 7.1: Core functional test cases verifying the expected operational workflow.

While all core functionalities were confirmed to operate correctly under these expected usage conditions, it is important to note the boundaries of this initial evaluation. Because the current testing protocol focused heavily on functional verification of the primary research workflow, future iterations of the software will require expanded edge-case and robust error-handling testing. This includes managing corrupted raw files, handling missing condition markers without system failure, and mitigating out-of-memory errors for exceptionally large cohorts.

7.2.3 Reproducibility Testing

Building upon the functional tests (specifically TC-06), the system was further evaluated for reproducibility by executing identical configurations multiple times. Results were consistently stable across runs, confirming that:

- Parameter configurations are strictly adhered to and applied uniformly.
- Cached results are reused appropriately, preventing redundant state changes.
- Visual and tabular outputs remain entirely stable across repeated executions of the same pipeline.

7.2.4 Machine Learning Evaluation Testing

The platform supports basic machine learning experimentation through its AI mode. While the current interface emphasizes regression-oriented outputs (loss, MAE, RMSE, and R^2), the underlying experiments were evaluated more broadly, including:

- **CCD-ITI approach:** Train on CCD-ITI segments and evaluate under a defined split strategy (preferably subject-independent).
- **Regression metrics (RT):** loss, MAE, RMSE, and R^2 .
- **Classification metrics (correctness):** accuracy, balanced accuracy, sensitivity, and specificity.

- **Verification:** Use SNR spectra (and related plots) to confirm steady-state characteristics before interpreting model outcomes.

7.2.5 Classification Runs

This section compares the effects of various data-balancing strategies on classification performance.

Balance Strategy	Acc	Bal Acc	Sens	Spec
None	0.729	0.501	0.993	0.009
Class Weight	0.604	0.544	0.674	0.414
Weighted Sampler	0.629	0.543	0.729	0.358
Undersample	0.652	0.537	0.787	0.287

Table 7.2: Selected classification runs from W&B export.

Interpretation: None shows majority-class collapse (almost all predictions biased to the positive class). Class Weight achieves better balance but still exhibits class-bias and limited generalization. Weighted Sampler helps recall but sacrifices specificity. Undersample improves minority exposure, though instability and false-positive pressure remain.

7.2.6 Regression Runs

This section summarizes regression performance across different loss functions.

Loss	MAE	RMSE	R ²	Pearson
MAE	0.2876	0.3743	0.0063	0.0840
MSE	0.2883	0.3749	0.0030	0.0562
Huber	0.2874	0.3740	0.0079	0.0903

Table 7.3: Regression runs from W&B export.

Interpretation: MAE shows low explanatory power with predictions remaining close to average RT. MSE exhibits similar stagnation, suggesting that loss choice alone does not improve structure learning. Huber

performs slightly better in correlation but still shows near-zero R^2 and weak trial-level fit.

Overall, no single run is presented as “best.” The current test report is used to identify concrete failure modes that guide the next iteration.

Interface Screenshots:

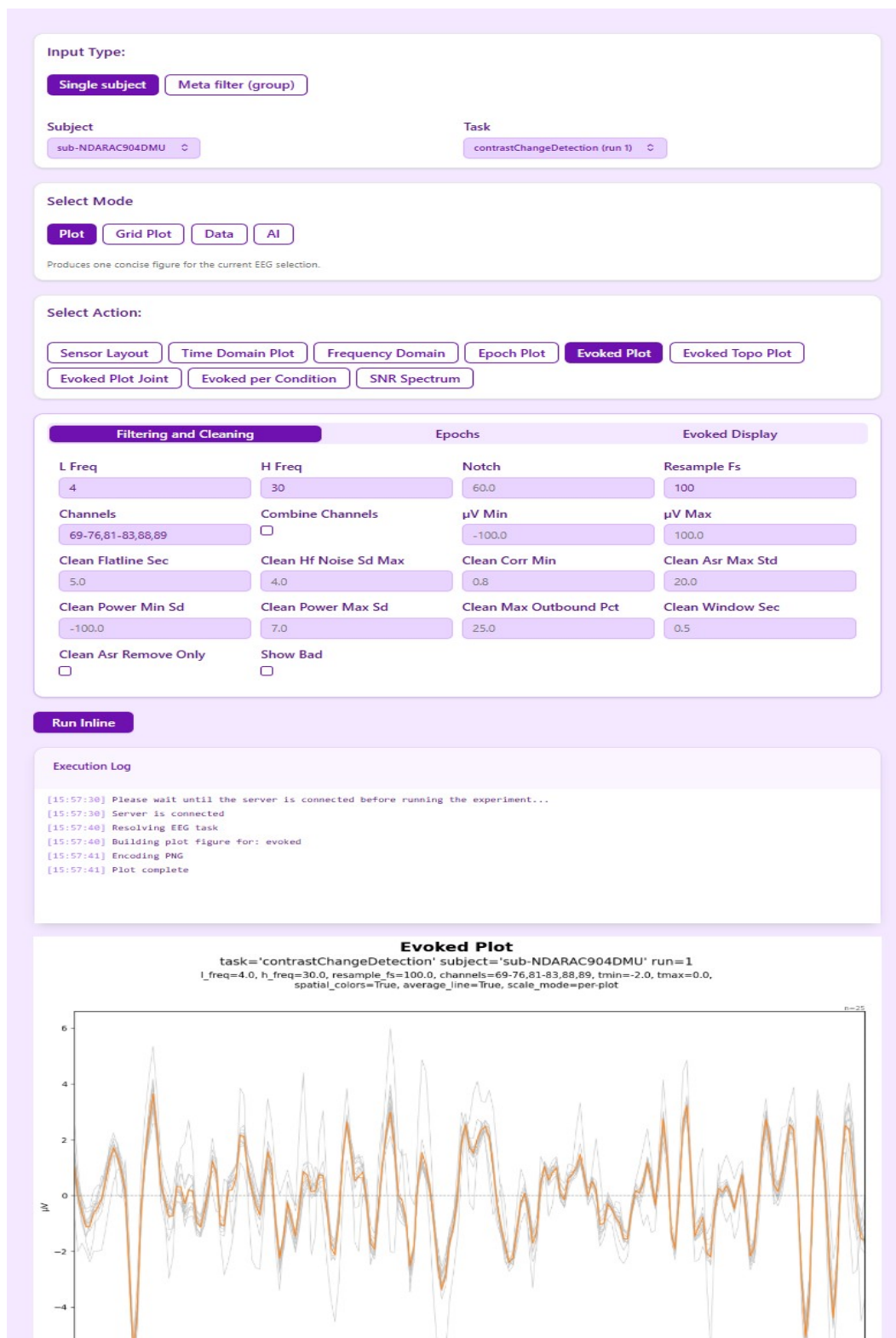


Figure 7.1: React Notebook Interface – Model visualization and execution for single subject

Input Type:

Single subject
Meta filter (group)

Task

contrastChangeDetection

Subject limit

☐ Per subject

Sex

Male

age_range

min
max

ehq_total_range

min
max

p_factor_range

min
max

attention_range

min
max

internalizing_range

min
max

externalizing_range

min
max

ccd_accuracy_range

min
max

ccd_response_time_range

min
max

Select Mode

Plot
Grid Plot
Data
AI

Produces one concise figure for the current EEG selection.

Select Action:

Sensor Layout
Time Domain Plot
Frequency Domain
Epoch Plot
Evoked Plot
Evoked Topo Plot
Evoked Plot Joint
Evoked per Condition
SNR Spectrum

Filtering and Cleaning		Epochs	
L Freq	H Freq	Notch	Resample Fs
4	30	60.0	100
Channels	Combine Channels	μV Min	μV Max
69-76,81-83,88,89	☐	-100.0	100.0
Clean Flatline Sec	Clean Hf Noise Sd Max	Clean Corr Min	Clean Asr Max Std
5.0	4.0	0.8	20.0
Clean Power Min Sd	Clean Power Max Sd	Clean Max Outbound Pct	Clean Window Sec
-100.0	7.0	25.0	0.5
Clean Asr Remove Only	Show Bad		
☐	☐		

Figure 7.2: React Notebook Interface – Model visualization and execution for multi subject

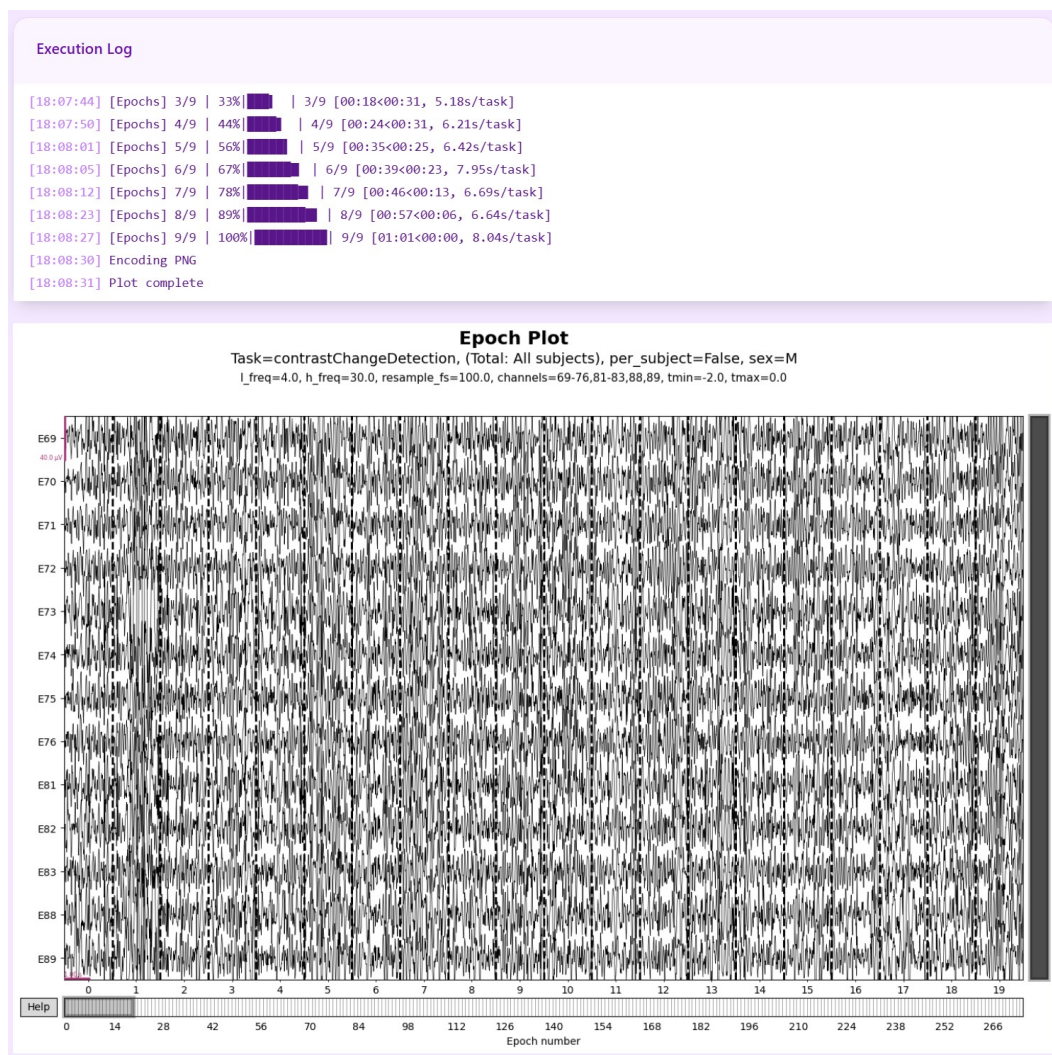


Figure 7.3: React Notebook Interface - model visualization and execution for multiple subjects (2)

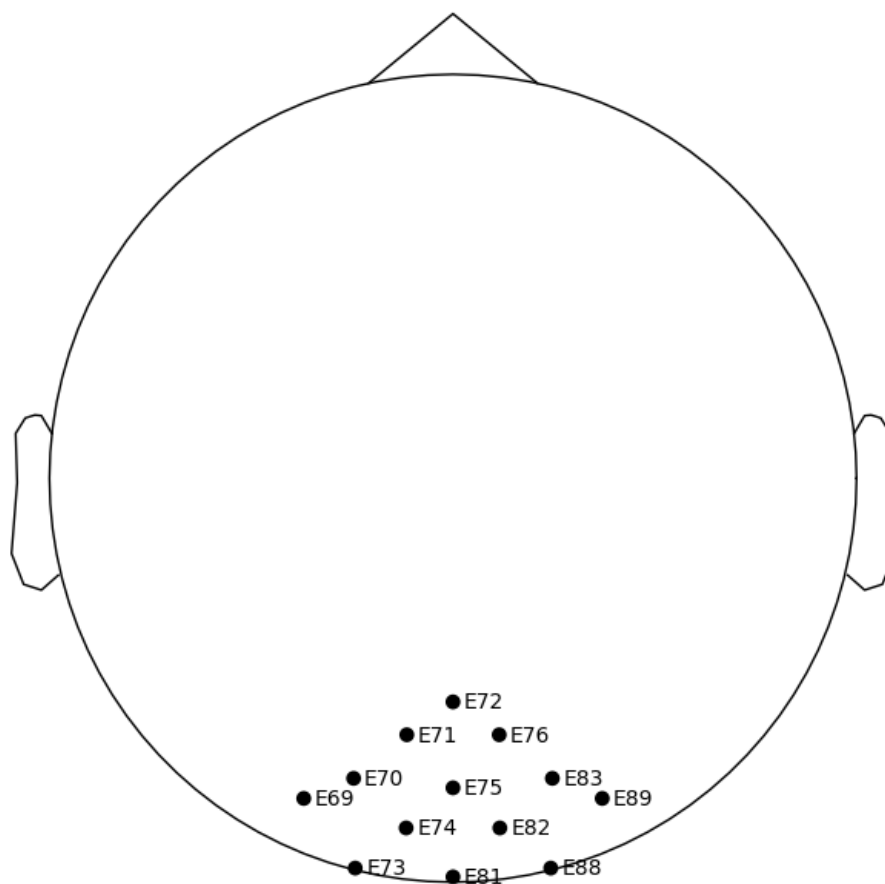


Figure 7.4: Sensor layout

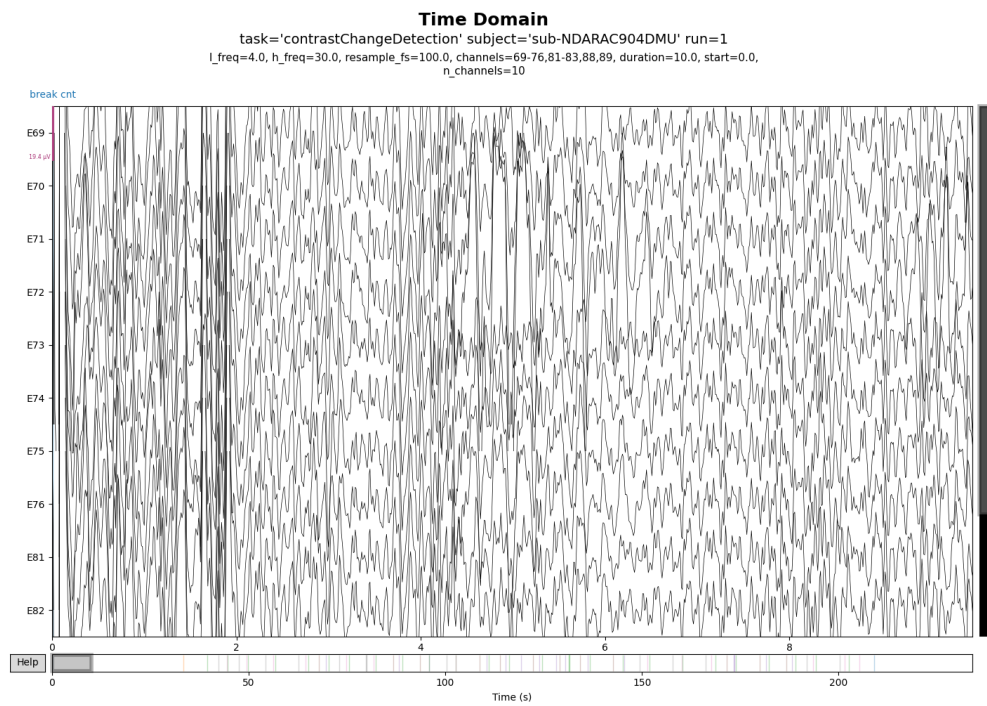


Figure 7.5: Time-domain signal

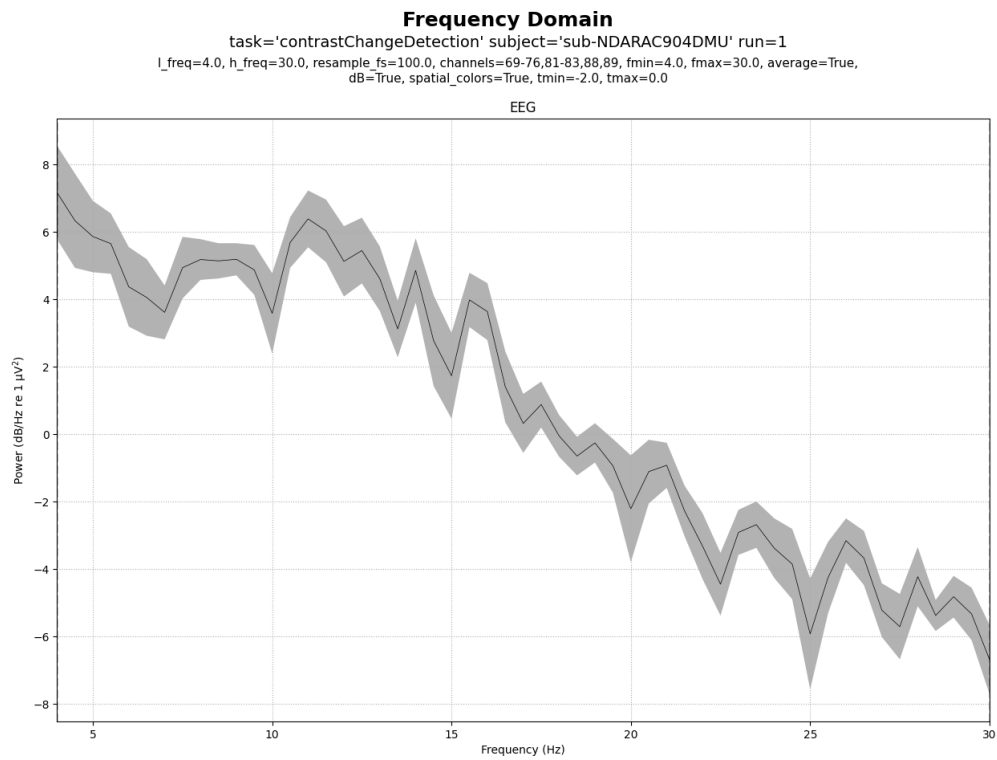


Figure 7.6: Frequency spectrum

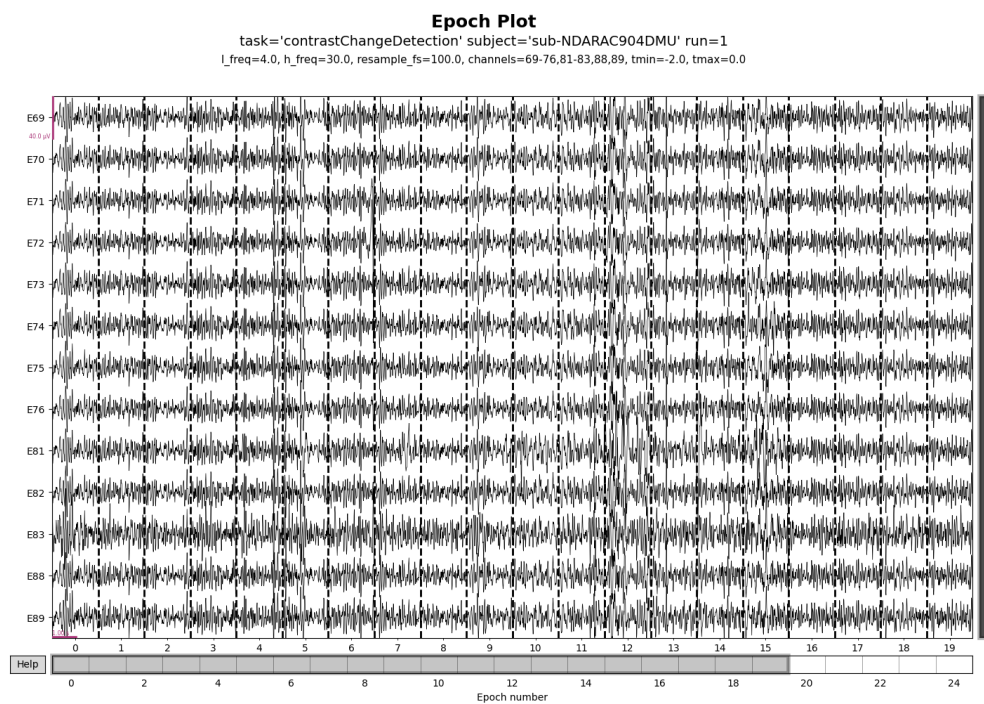


Figure 7.7: Epoch visualization

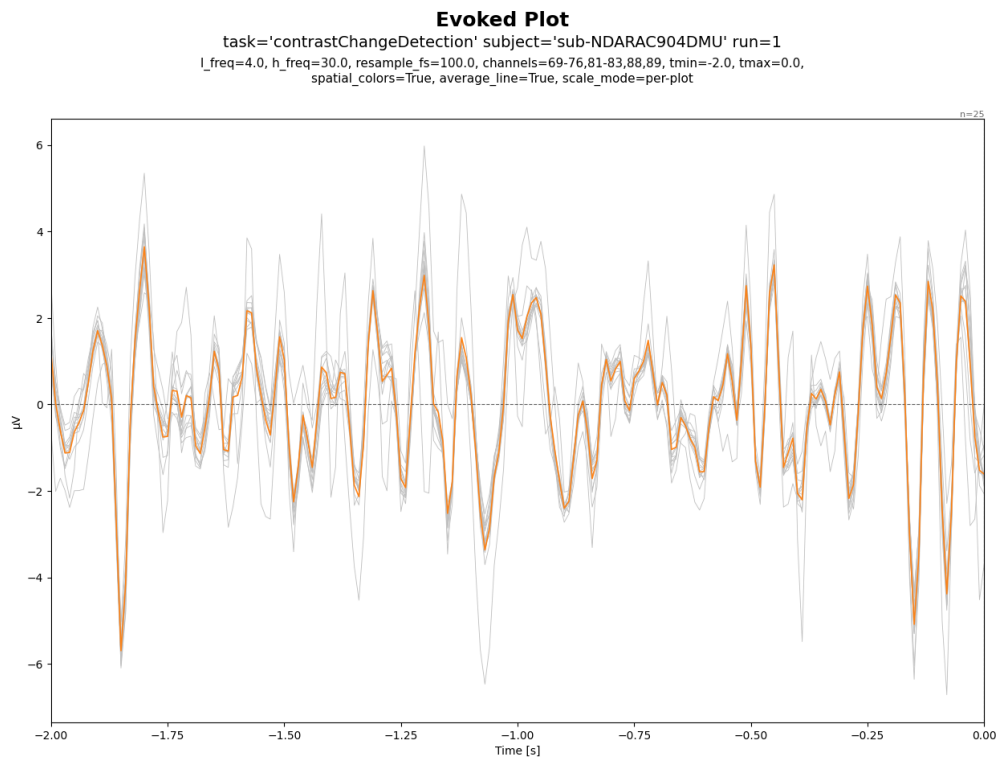


Figure 7.8: Evoked response

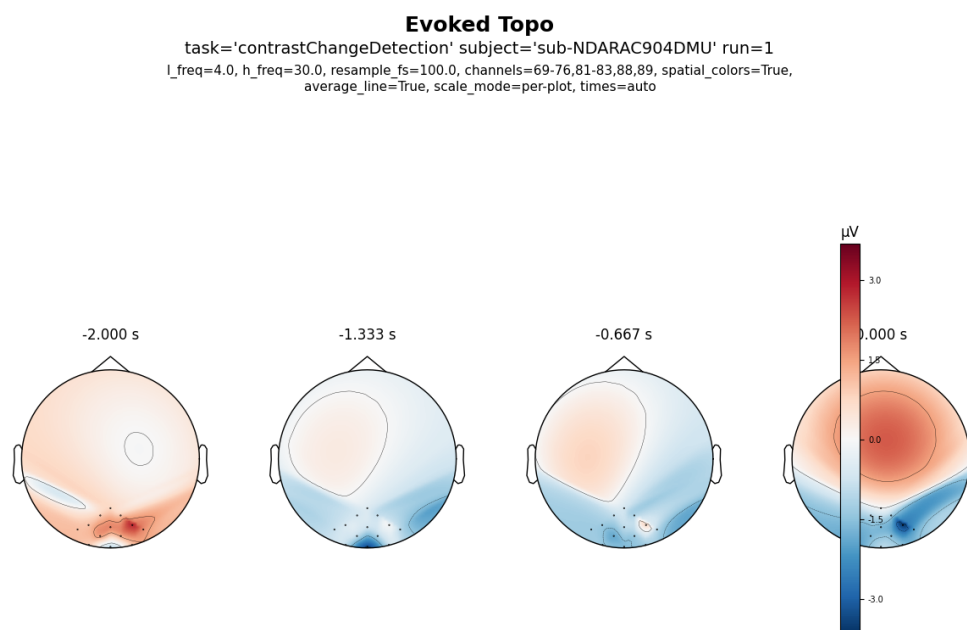


Figure 7.9: Evoked topomap

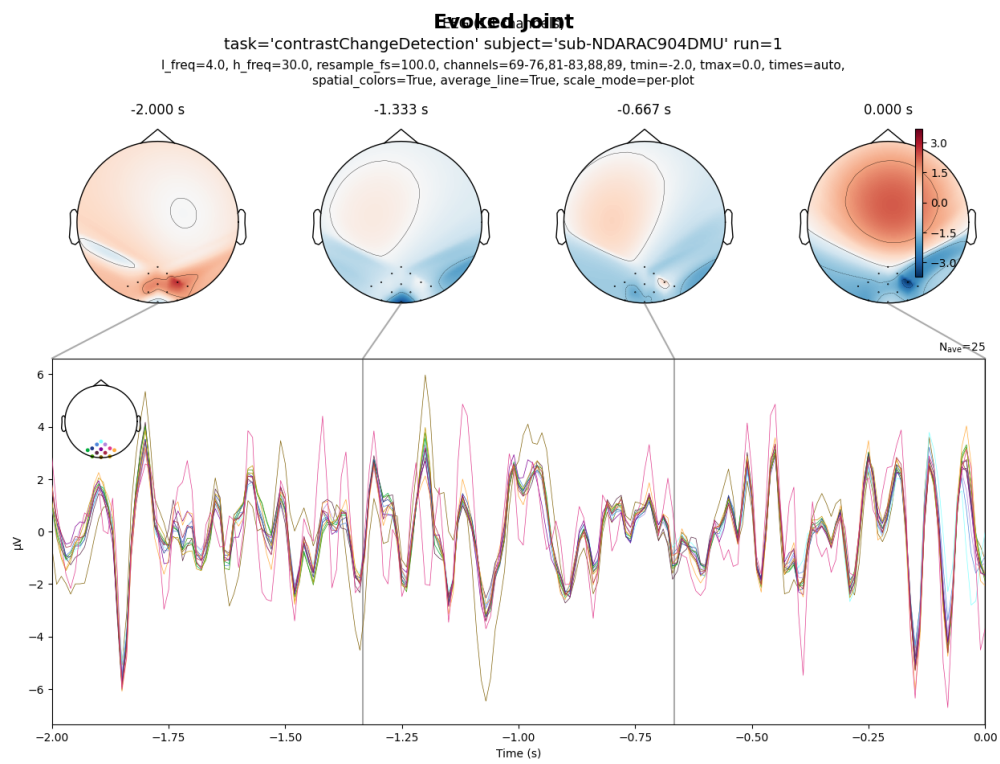


Figure 7.10: Evoked joint plot

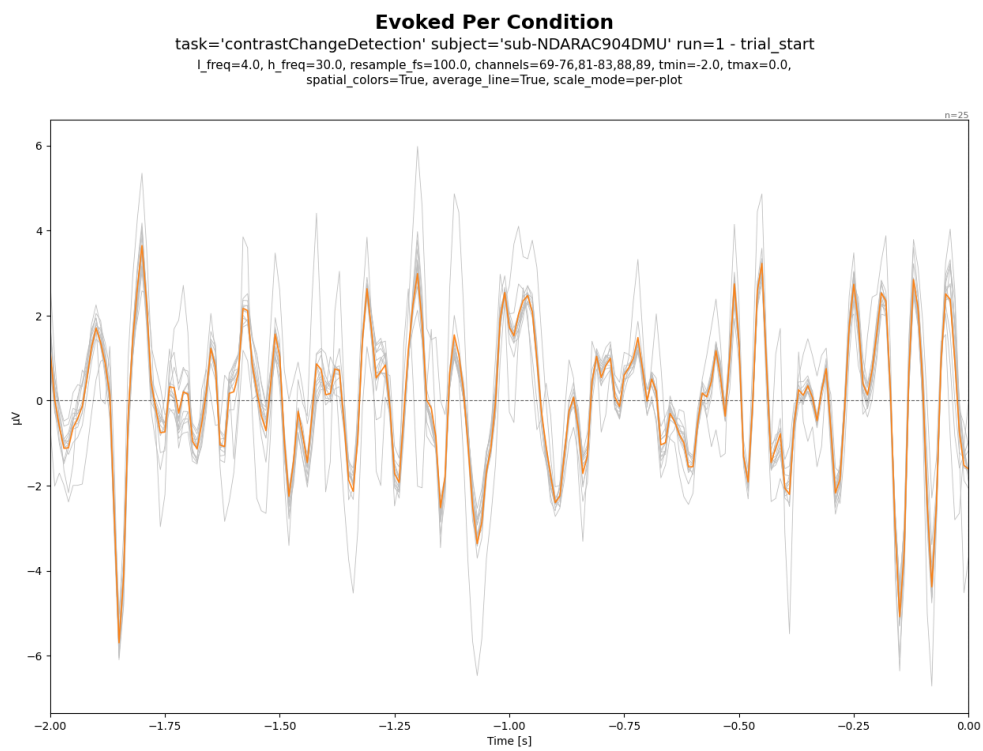


Figure 7.11: Evoked per condition

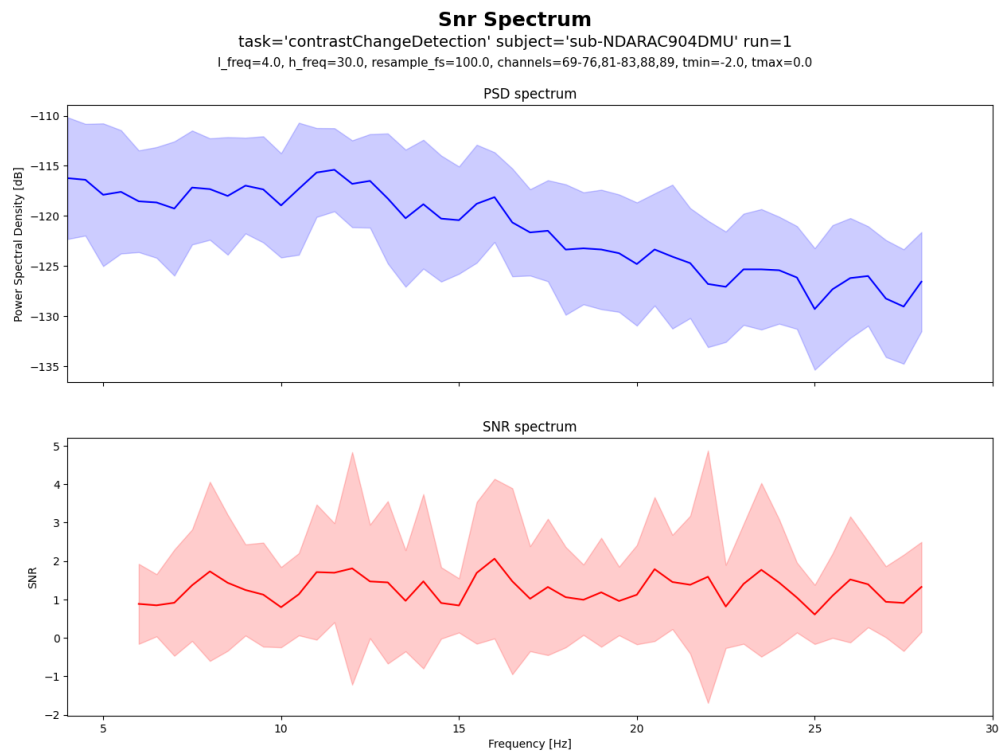


Figure 7.12: Signal-to-noise ratio spectrum

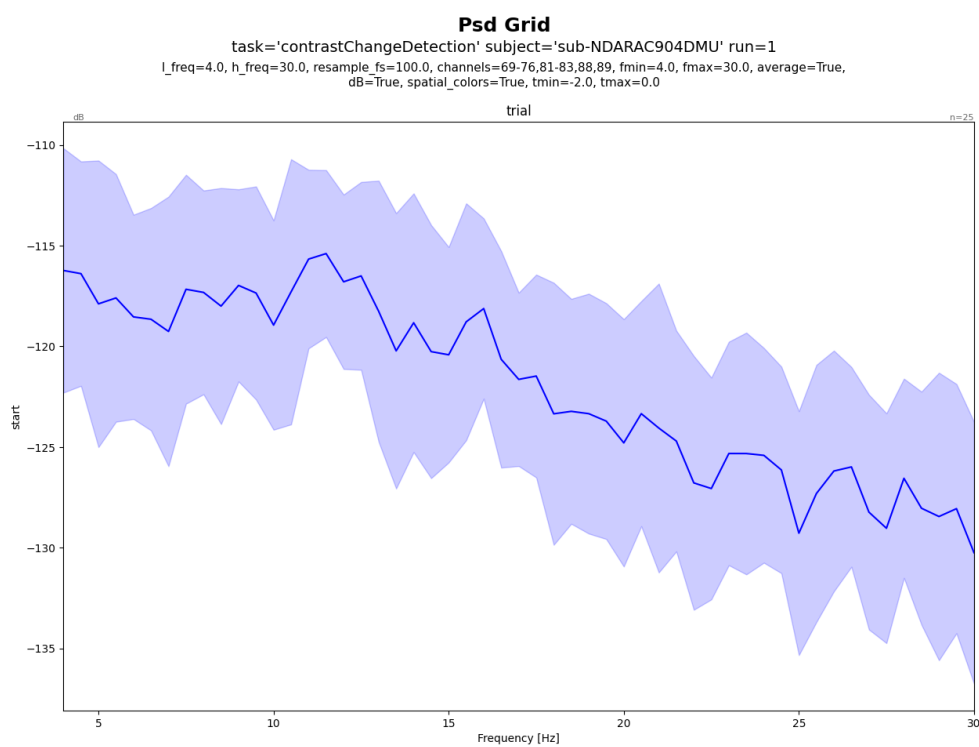


Figure 7.13: Power spectral density grid

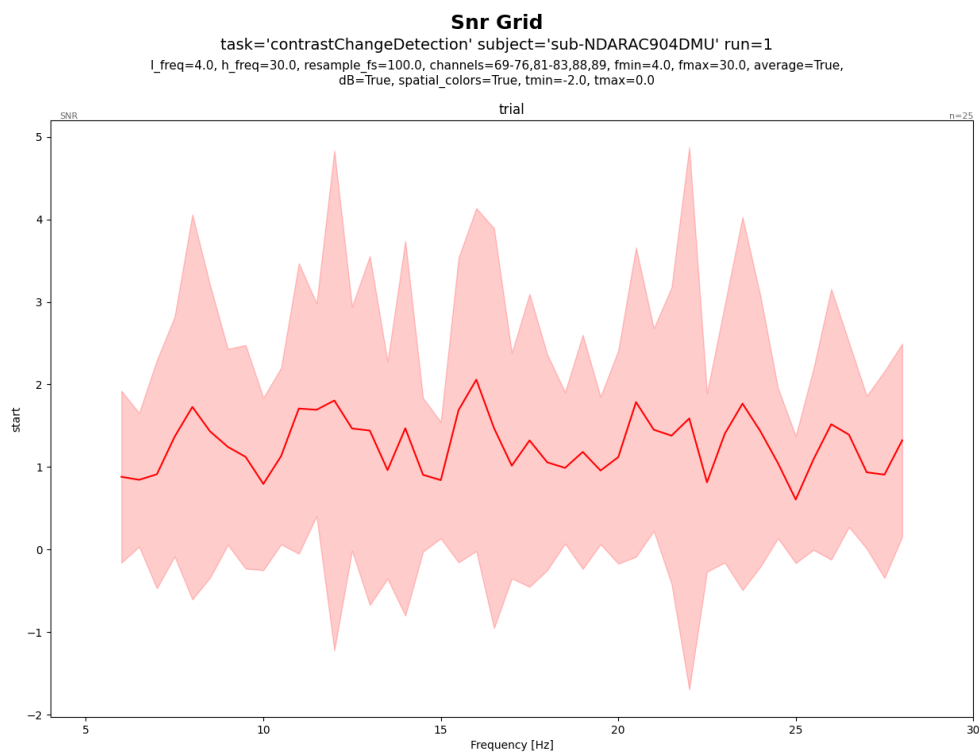


Figure 7.14: Signal-to-noise ratio grid

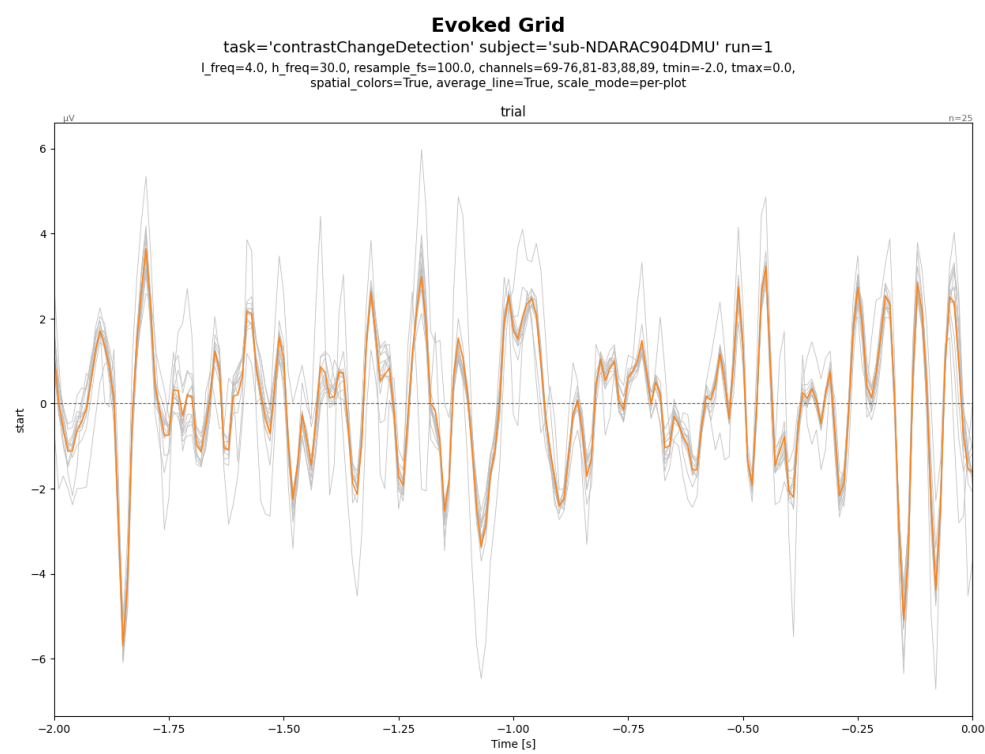


Figure 7.15: Evoked grid view

onset	duration	sample	value	event_code	feedback
0		0	break cnt	break cnt	
33.400		16,700	contrastChangeB1_start	94	
39.484		19,742	contrastTrial_start	5	
42.284		21,142	right_target	9	
44.414		22,207	right_buttonPress	13	smiley_face
44.684		22,342	contrastTrial_start	5	
47.484		23,742	right_target	9	
49.444		24,722	right_buttonPress	13	smiley_face
49.884		24,942	contrastTrial_start	5	
54.284		27,142	right_target	9	

Figure 7.16: EEG Table

label	n_epochs	n_channels	timespan_sec	sampling_rate	duration_per_epoch_sec
trial_start	25	128	2	100	2.010

Figure 7.17: Epochs Table

field	value
PowerLineFrequency	60
TaskName	contrastChangeDetection
EEGChannelCount	129
EEGReference	Cz
RecordingType	continuous
RecordingDuration	234.104
SamplingFrequency	500
SoftwareFilters	n/a

Figure 7.18: Metadata Table

n_epochs	n_channels	n_times	sfreq	label::93.0556
25	13	201	100	25

Figure 7.19: Build Dataset Table

epoch	train_loss	val_loss	val_mae	val_r2
1	8785.288	8644.565	92.976	—
2	7657.540	8628.153	92.888	—
3	7029.752	8612.935	92.806	—
4	6579.648	8599.336	92.733	—
5	6160.696	8579.167	92.624	—
6	5748.555	8556.205	92.500	—
7	5390.962	8517.980	92.293	—
8	4998.942	8471.551	92.041	—
9	4629.825	8418.859	91.754	—
10	4231.050	8354.287	91.402	—
11	3862.044	8270.908	90.945	—
12	3482.110	8162.792	90.348	—
13	3158.336	8036.194	89.645	—
14	2826.153	7879.874	88.769	—
15	2488.775	7690.543	87.696	—
16	2187.789	7486.270	86.523	—
17	1870.673	7235.068	85.059	—
18	1601.797	6951.105	83.373	—
19	1377.668	6628.659	81.417	—
20	1162.443	6287.875	79.296	—
21	964.768	5937.247	77.054	—
22	750.151	5559.127	74.560	—
23	605.925	5176.067	71.945	—
24	465.719	4776.360	69.111	—
25	356.632	4384.486	66.215	—
26	277.896	3997.530	63.226	—
27	189.174	3634.381	60.286	—
28	133.263	3285.107	57.316	—
29	84.388	2969.587	54.494	—
30	45.871	2679.816	51.767	—
31	26.688	2415.238	49.145	—
32	18.966	2180.149	46.692	—
33	13.227	1955.059	44.216	—
34	14.910	1780.155	42.192	—
35	25.970	1598.232	39.977	—
36	16.541	1450.283	38.082	—
37	28.291	1308.515	36.172	—
38	28.556	1169.425	34.197	—
39	43.122	1056.393	32.501	—
40	43.380	959.707	30.978	—
41	46.487	869.578	29.487	—
42	39.488	766.095	27.677	—
43	55.277	680.290	26.080	—
44	33.083	620.142	24.900	—
45	32.417	539.505	23.225	—
46	28.597	490.384	22.142	—
47	23.332	427.502	20.675	—
48	12.073	383.657	19.584	—
49	11.887	339.250	18.417	—
50	10.848	305.535	17.474	—

Figure 7.20: Train EEGMultiNetRegression Table

Layer	Output Shape	Params
EEGNetMultiReg	[1, 1]	100,977
Conv2d	[1, 16, 13, 138]	1,040
BatchNorm2d	[1, 16, 13, 138]	0
ZeroPad2d	[1, 16, 14, 171]	0
Conv2d	[1, 32, 13, 140]	32,800
BatchNorm2d	[1, 32, 13, 140]	0
MaxPool2d	[1, 32, 6, 35]	0
ZeroPad2d	[1, 32, 13, 38]	0
Conv2d	[1, 64, 6, 35]	65,600
BatchNorm2d	[1, 64, 6, 35]	0
MaxPool2d	[1, 64, 3, 8]	0
Linear	[1, 1]	1,537
TOTAL PARAMS		100,977
TRAINABLE PARAMS		100,977
NON-TRAINABLE PARAMS		0
TOTAL MULT-ADDS (M)		75.340
INPUT SIZE (MB)		0.010
FWD/BWD SIZE (MB)		0.800
PARAM SIZE (MB)		0.400

Figure 7.21: Model Summary Table

Chapter 8

Project Defense and Project Expo

This chapter presents the project evaluation and dissemination processes conducted through two rounds of project defense and participation in the SKE Project Expo. This chapter documents the feedback received from evaluators, the revisions made in response to that feedback, and the outcomes of those revisions. In addition, it discusses the team's experience in presenting the project to a broader audience, including poster design, audience interaction, and reflections on how the Expo contributed to the overall development of the project.

8.1 First Project Defense

8.1.1 Comments and Feedback from the Evaluators

During the first project defense, the evaluators observed that the project was primarily presented from a research-oriented perspective. While this direction was valid at the early stage of development, it led to some ambiguity regarding the overall purpose and positioning of the project.

In particular, the evaluators found that the intended target users were not clearly defined. It was unclear whether the system was designed for researchers, developers, or general users. This made it difficult to determine how the system would be used in practice and what its main contribution was.

The evaluators also suggested that the team should provide a clearer explanation of what the project is doing and what research problem it is addressing. Although technical details were presented, especially in relation to EEG analysis and machine learning, the connection between these components and the overall objective was not sufficiently clear.

As a result, both the research motivation and the system's role were not communicated effectively.

Furthermore, the system interface appeared highly technical and resembled a “cockpit-style” design, which reduced its accessibility. This reinforced the perception that the project was more research-focused rather than a user-oriented software system.

Overall, the main concern raised during the first defense was that the project was still too research-oriented in its presentation, with unclear target users and insufficient explanation of its practical purpose.

8.1.2 Revisions Required, Actions Taken, and Outcomes

Following the first project defense, the team recognized that the main issue was the lack of clarity in project positioning, particularly in terms of target users and overall purpose. However, at this stage, the project was still in an early phase of development, and its research-oriented nature was still appropriate.

Instead of making immediate structural changes, the team focused on reflecting on the feedback and reassessing the direction of the project. This included considering how the system could evolve from a research prototype into a more complete and user-oriented software platform, as well as how to better communicate its objectives and contributions.

As a result, the first defense served as an important turning point by highlighting the gap between technical implementation and clear communication. Although no major revisions were implemented immediately, the insights gained during this stage directly informed the significant changes made prior to the second project defense.

8.2 Second Project Defense

8.2.1 Comments and Feedback from the Evaluators

In the second project defense, the evaluators observed that the project had improved in terms of overall clarity and structure. The revised presentation made it easier to understand the purpose of the system and

how its components are connected. The project was also recognized as being conceptually valid and implementable, indicating that the core idea had been successfully demonstrated.

However, the evaluators noted that the presentation was still relatively technical and research-heavy, which could limit accessibility for a broader audience. While the direction of the project was clearer than in the first defense, further improvements in simplifying explanations and emphasizing practical usage were recommended.

In addition, the evaluators raised questions regarding the system architecture, data processing pipeline, and potential real-world applications. These questions reflected a growing interest in understanding not only the technical components, but also how the system operates as a complete solution and how it could be applied in practical contexts.

8.2.2 Additional Revisions, Actions Taken, and Final Outcomes

The main revision involved redefining the project as an interactive EEG analysis platform, rather than solely a machine learning experiment. This shift led to several concrete changes.

First, the system design was adjusted to emphasize modularity and usability, allowing users to interact more effectively with preprocessing, visualization, and modeling components. Second, the presentation narrative was restructured to highlight the overall workflow and system architecture, instead of focusing primarily on model-level details.

In addition, the target users of the system were more clearly defined as researchers and practitioners working with EEG data, which helped align both the system design and the presentation with practical use cases.

As a result of these changes, the project became more aligned with software engineering objectives. The system's purpose—supporting reproducible EEG analysis and experimentation—was communicated more clearly. This revision significantly improved the coherence of the project and demonstrated a more mature and well-defined direction.

8.2.3 Suggestions on How Project Defense Can Better Support Software Development

The project defense process played an important role in guiding the development of the system, particularly in clarifying the project's direction and improving its communication. Feedback from the evaluators helped the team recognize that technical implementation alone is not sufficient; clear explanation of objectives, target users, and system usage is equally important.

However, the process could be further improved to better support software development. For example, providing earlier feedback on project scope and positioning would help reduce ambiguity during the initial stages. Additionally, clearer evaluation criteria that distinguish between research contributions and software engineering aspects would allow students to better align their work with expectations.

Furthermore, emphasizing usability, system architecture, and real-world applicability during evaluation would encourage teams to develop more practical and user-oriented solutions. Strengthening these aspects would make the defense process more effective in supporting the development of complete and impactful software systems.

8.3 Project Expo Participation

8.3.1 Poster Presentation and Poster Components

The poster presented at the Project Expo was designed to communicate the project as an integrated EEG analysis platform. It was structured into several key sections, including project overview, objectives, system architecture, data pipeline, platform visualization, study setup, predictive models, research findings, and future work.

The main objective of the poster was to convey, within a short time, that the project provides a unified and reproducible workflow for EEG data analysis and prestimulus behavioral prediction. To achieve this, the design emphasized a balance between technical content and visual clarity. Diagrams illustrating the system architecture and data pipeline were used

to provide an intuitive understanding of how the platform operates, while concise text descriptions summarized the key components and findings.

The poster also highlighted both aspects of the project: the software platform and the underlying machine learning models. This ensured that the audience could understand not only how the system functions, but also what insights it produces.

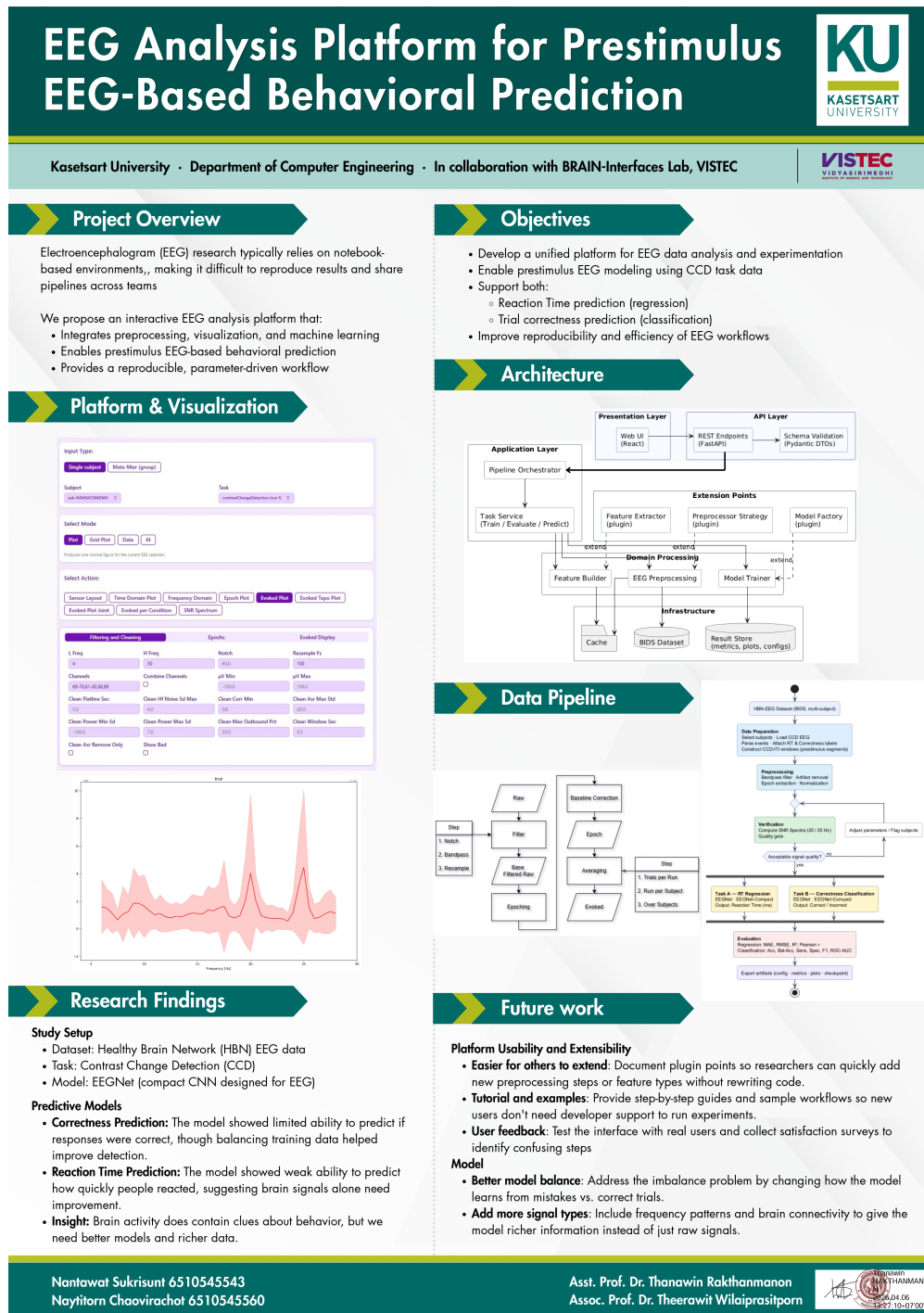


Figure 8.1: Poster

8.3.2 Event Photos and Awards Received

During the Project Expo, the team actively participated by presenting the system at their assigned booth. A live demonstration of the platform was conducted, allowing visitors to observe the workflow and interact with the system in real time. This approach helped communicate the practical functionality of the project more effectively.

The team did not receive any awards during the event. However, the experience of presenting and demonstrating the system provided valuable opportunities for engagement and feedback from a diverse audience.

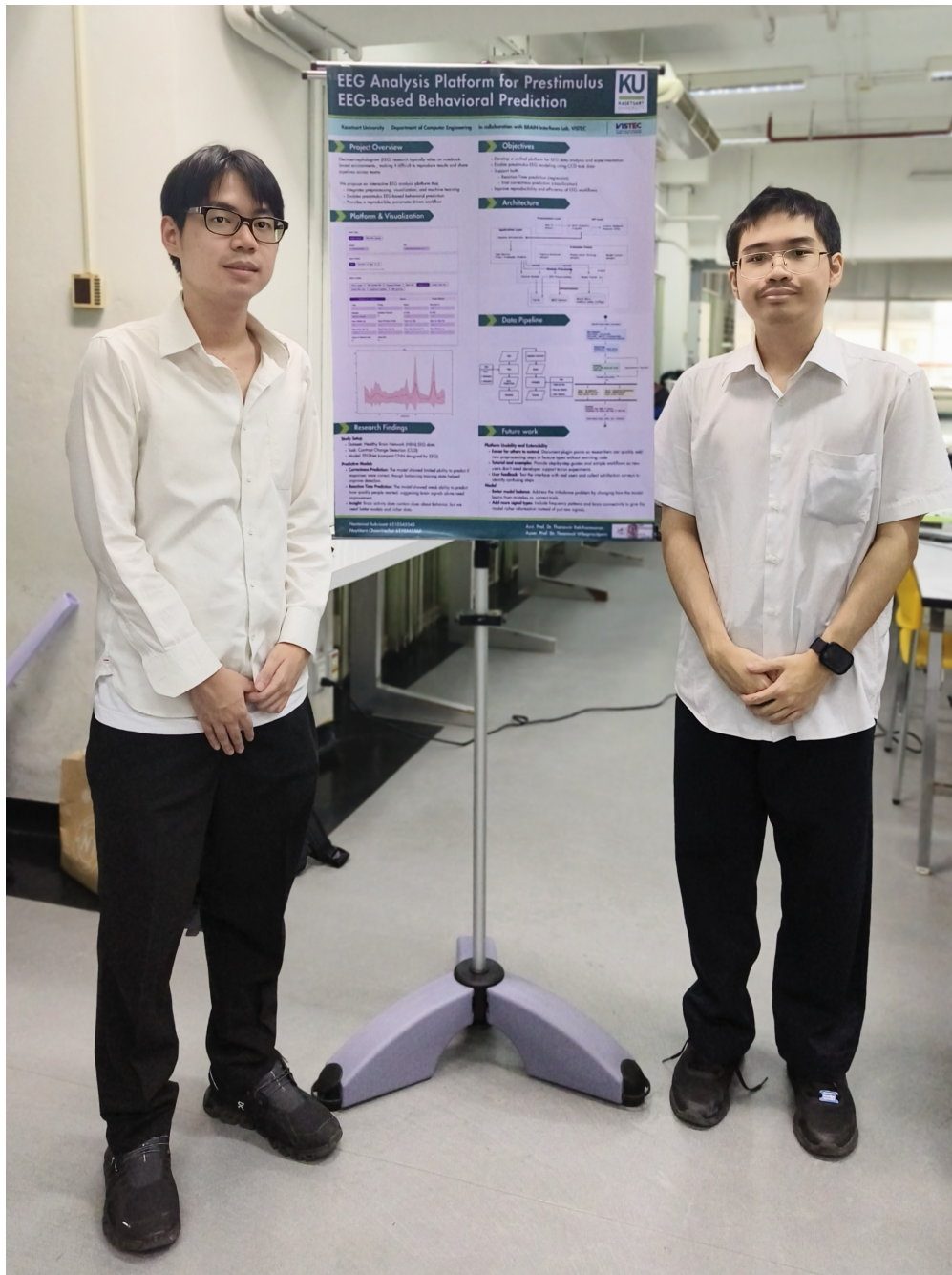


Figure 8.2: Event Photo

8.3.3 Questions Raised During the Expo and the Team's Responses

During the Expo, visitors raised several questions regarding the technical details and practical applications of the project. One common question concerned the source of the data used in the system. The team explained that the platform utilizes EEG data from the Healthy Brain Network dataset, which is widely used in research.

Another frequently asked question was about the type of artificial intelligence used. The team described the use of EEGNet, a convolutional neural network designed specifically for EEG signal analysis, and explained its role in both regression and classification tasks.

Visitors also asked about the system architecture and data processing pipeline. In response, the team explained the workflow, including data preprocessing, model execution, and visualization, using diagrams provided in the poster.

In addition, questions were raised about potential real-world applications of the system. The team discussed possible use cases such as behavioral prediction, cognitive state analysis, and support tools for EEG-based research.

These interactions highlighted the importance of clearly explaining both the technical and practical aspects of the system, and they provided valuable external perspectives on the project.

8.3.4 Recommendations for Improving the Expo

Overall, the Project Expo was well-organized and provided a valuable platform for presenting student projects. The event facilitated interaction between teams and a broad audience, enabling knowledge sharing and constructive discussions.

While no major issues were identified, there are opportunities for improvement. Increasing the involvement of industry representatives could provide additional insights into real-world applications and enhance the practical relevance of the projects. Additionally, more structured feedback

mechanisms could help participants better understand their strengths and areas for improvement.

Chapter 9

Knowledge and Skills Evaluation

This chapter evaluates the knowledge and skills developed throughout the project by comparing them with the Program Learning Outcomes (PLOs) and the expected rubric levels established by the curriculum. The evaluation is conducted from three perspectives: AI-based assessment, student self-assessment, and advisor assessment. The purpose of this chapter is not only to measure achievement, but also to provide reflective insight into how different evaluative perspectives reveal strengths, gaps, and opportunities for further development.

9.1 Comparison Between Program Targets and AI-Based Evaluation

The first part focuses on comparing the program's expected learning outcomes with the evaluation generated by AI. The AI assigns an evaluation level for each dimension and provides justification explaining why each level was assigned.

9.1.1 Executive Summary

This project demonstrates a strong integration of software engineering and AI experimentation practices, particularly in building a reproducible EEG analysis platform. The student shows clear competency in system design, pipeline engineering, and experimental rigor, but limited success in achieving strong AI model performance.

Overall, the student meets most intended graduate outcomes for an undergraduate Software Engineering program, especially in structured system design, reproducibility, experimentation workflows, and technical communication. However, gaps remain in advanced AI effectiveness and

system scalability, preventing full alignment with higher-level expectations in some dimensions.

9.1.2 Detailed Evaluation Table

Table 9.1 summarizes the AI-assessed level for each dimension relative to the expected curriculum level.

Table 9.1: AI-based evaluation of knowledge and skills dimensions.

Dimension	AI-Assessed Level Description	Expected Level	AI Level	Judgment
Software Development & Programming	Scalable systems, modular architecture, deployment	2	3	Exceeds
Machine Learning, LLM & AI Systems	Task-specific ML/LLM systems	3	2	Below
Data Management & Knowledge Systems	Scalable knowledge systems with governance	2	3	Exceeds
Critical & Creative Thinking	Hypothesis-driven, iterative, metric-based	3	3	Meets
Collaboration & Communication	Cross-disciplinary leadership, evidence-based communication	3	3	Meets

9.1.3 Justification for Each Dimension

Software Development & Programming — Level 3 (Exceeds Expectations)

The AI assigned Level 3 based on the following evidence:

- The system uses a layered architecture (React + FastAPI + pipeline design), demonstrating modular design.
- Implementation includes multiple design patterns (Pipeline, Strategy, Factory, Registry, Observer), indicating architectural maturity.

- The system supports caching, reproducibility, and parameter-driven execution, aligning with scalable engineering practices.
- Use of Docker deployment and a hybrid architecture (cloud and local backend) shows real-world system considerations.
- Presence of traceability, logging, and job artifacts reflects evaluation-driven development.

Level 4 was not assigned because there is no evidence of large-scale production deployment, DevOps pipelines, or organizational engineering standards. The system does not address concerns such as security, high availability, or distributed orchestration.

Machine Learning, LLM & AI Systems — Level 2 (Below Expectations)

The AI assigned Level 2 based on the following evidence:

- The student builds task-specific ML pipelines for reaction time regression and correctness classification.
- Proper evaluation metrics are used (MAE, R^2 , F1, balanced accuracy).
- Data splitting strategies (subject-independent) are implemented to avoid leakage.
- Class imbalance is addressed using weighting and sampling techniques.
- A reproducible ML experimentation workflow with logging and validation is established.

Level 3 was not assigned because there is no evidence of production ML systems, monitoring or observability, model lifecycle management, or automated pipelines (MLOps). Furthermore, model performance remains weak ($R^2 \approx 0$, imbalance issues unresolved), and no advanced architectures or optimization strategies were explored in depth.

Data Management & Knowledge Systems — Level 3 (Exceeds Expectations)

The AI assigned Level 3 based on the following evidence:

- Structured data pipelines are implemented from raw EEG through processed data to features.
- Caching with parameter hashing and pipeline versioning is used.
- Traceability is ensured via job specifications, logs, and reproducible outputs.
- Cohort filtering and multi-subject analysis support scalable data handling.
- Data integrity checks, including label alignment validation and fail-fast mechanisms, are maintained.

Level 4 was not assigned because there are no enterprise-level features such as federated systems, policy-based governance, or cross-system integration.

Critical & Creative Thinking — Level 3 (Meets Expectations)

The AI assigned Level 3 based on the following evidence:

- An explicit hypothesis-driven approach is used: predicting behavior from prestimulus EEG.
- An iterative refinement process is applied, including fixing trial alignment, improving preprocessing, and adjusting splits.
- Strong emphasis is placed on validation before modeling (SNR verification).
- Failure modes are clearly identified, including regression collapse and class imbalance.
- Reflective analysis acknowledges limitations and future work.

Level 4 was not assigned because there is no evidence of novel methodology or new research contributions, and no new frameworks or significantly innovative solutions are proposed.

Collaboration & Communication — Level 3 (Meets Expectations)

The AI assigned Level 3 based on the following evidence:

- The report is well-structured and comprehensive, covering requirements, design, implementation, and evaluation.
- Traceability between user stories and requirements is included.
- Technical risks and limitations are clearly communicated.
- Detailed diagrams, tables, and system explanations are provided.
- Alignment between system design and research goals is demonstrated.

Level 4 was not assigned because there is no evidence of leading teams, cross-disciplinary coordination, or influencing broader communities.

9.1.4 Gap Analysis

Table 9.2: Gap analysis between expected curriculum levels and AI-assessed levels.

Dimension	Expected Level	AI Level	Gap
Software Development & Programming	2	3	Exceeds
Machine Learning, LLM & AI Systems	3	2	Below
Data Management & Knowledge Systems	2	3	Exceeds
Critical & Creative Thinking	3	3	Meets
Collaboration & Communication	3	3	Meets

In summary, the student exceeds expectations in Software Development and Data Management, meets expectations in Critical Thinking and Collaboration, and falls below expectations in Machine Learning.

9.1.5 AI Final Conclusion

Strengths identified by the AI evaluation:

- Strong software engineering foundation with modular, scalable architecture.
- Excellent reproducibility and traceability practices.
- Rigorous experimental methodology and critical reasoning.
- Clear and structured technical communication.

Limitations identified by the AI evaluation:

- Weak AI model performance and optimization.
- Limited exploration of advanced ML techniques or architectures.
- No evidence of production-grade ML systems (MLOps).

The student is well-prepared for roles in software engineering and AI system development, particularly in research engineering, data pipeline design, and experimentation platforms. Further development is needed in advanced machine learning techniques and model performance optimization.

9.1.6 Radar Chart Comparing Program Targets and AI Evaluation

Figure 9.1 presents a radar chart comparing the program's expected levels against the AI-assessed levels across all five dimensions (scale 1–4).

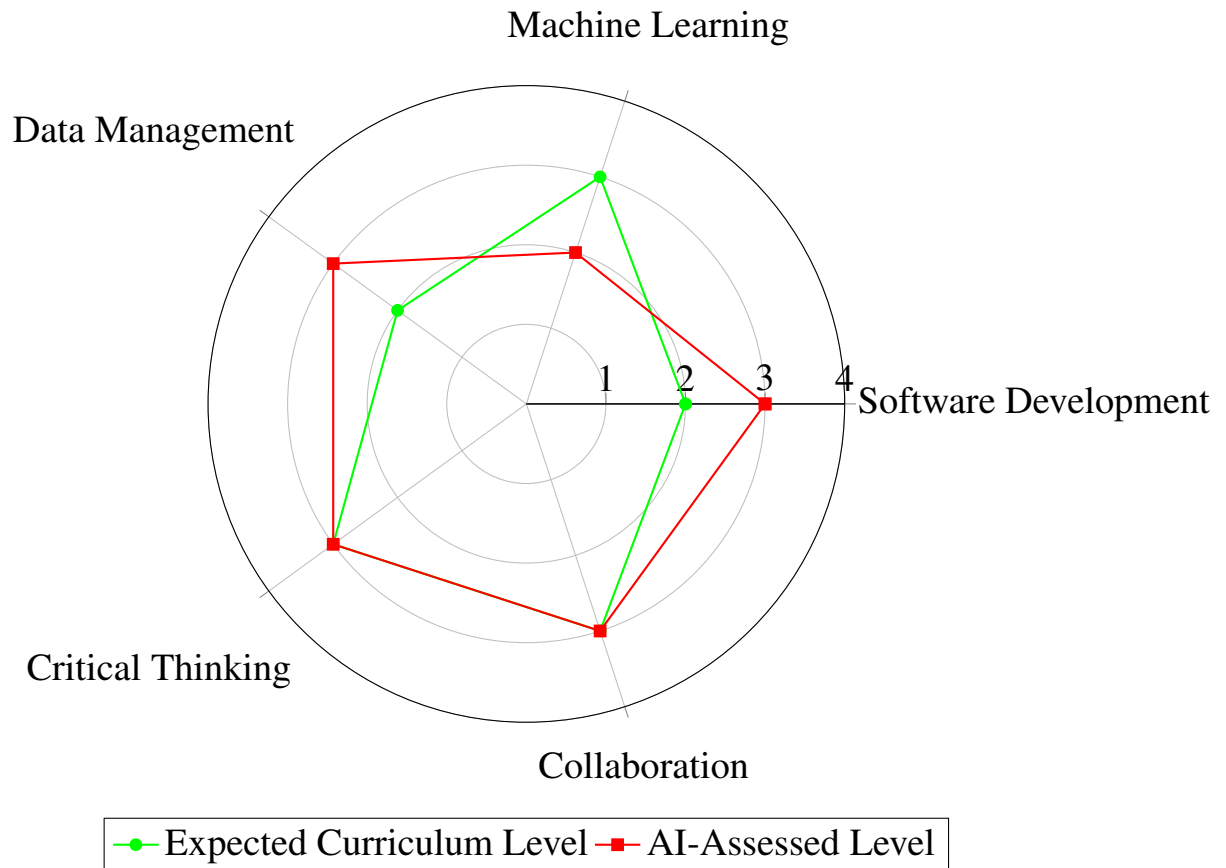


Figure 9.1: Radar chart comparing expected curriculum levels and AI-assessed levels across five evaluation dimensions.

9.1.7 Reflection on the AI Evaluation

Overall, we agree with most parts of the AI evaluation, especially the comments on software architecture, reproducibility, and technical communication. However, we believe the Machine Learning score may be somewhat conservative when interpreted without project context.

Our work attempts a difficult and less explored objective: predicting behavioral outcomes from prestimulus EEG signals in a CCD setting. This problem has a high uncertainty level and substantial research and engineering risk. It often requires longer research timelines in graduate-level contexts, whereas this project was conducted at undergraduate level

within less than one year. As a result, substantial effort was required to build reliable data alignment, preprocessing, and leakage-safe evaluation before meaningful model training was possible.

As a result, a significant portion of project time was invested in foundational research engineering work that is not always visible through final model metrics alone. In particular, we prioritized trial-label integrity, subject-independent splits, and reproducible experiment tracking to ensure that any conclusion is scientifically valid. These are essential requirements for trustworthy AI experimentation, even when predictive performance remains limited.

Therefore, we accept that the current model performance has not yet reached the target level, but we also argue that the project demonstrates strong competence in designing and executing an end-to-end AI experimentation framework for a challenging novel problem. In this sense, the main limitation is outcome maturity, not lack of methodological rigor or effort.

9.2 Self-Evaluation by the Student

The second part focuses on student self-evaluation. The student assesses their own knowledge and skills and compares that self-assessment with the AI-generated evaluation. The discussion explains why the two evaluations may differ.

9.2.1 Radar Chart Comparing Program Targets, AI Evaluation, and Self-Evaluation

This section includes a radar chart with three lines in the same figure: (1) the program's target levels, (2) the AI-based evaluation, and (3) the student's self-evaluation. This figure visually illustrates similarities and differences among the three perspectives.

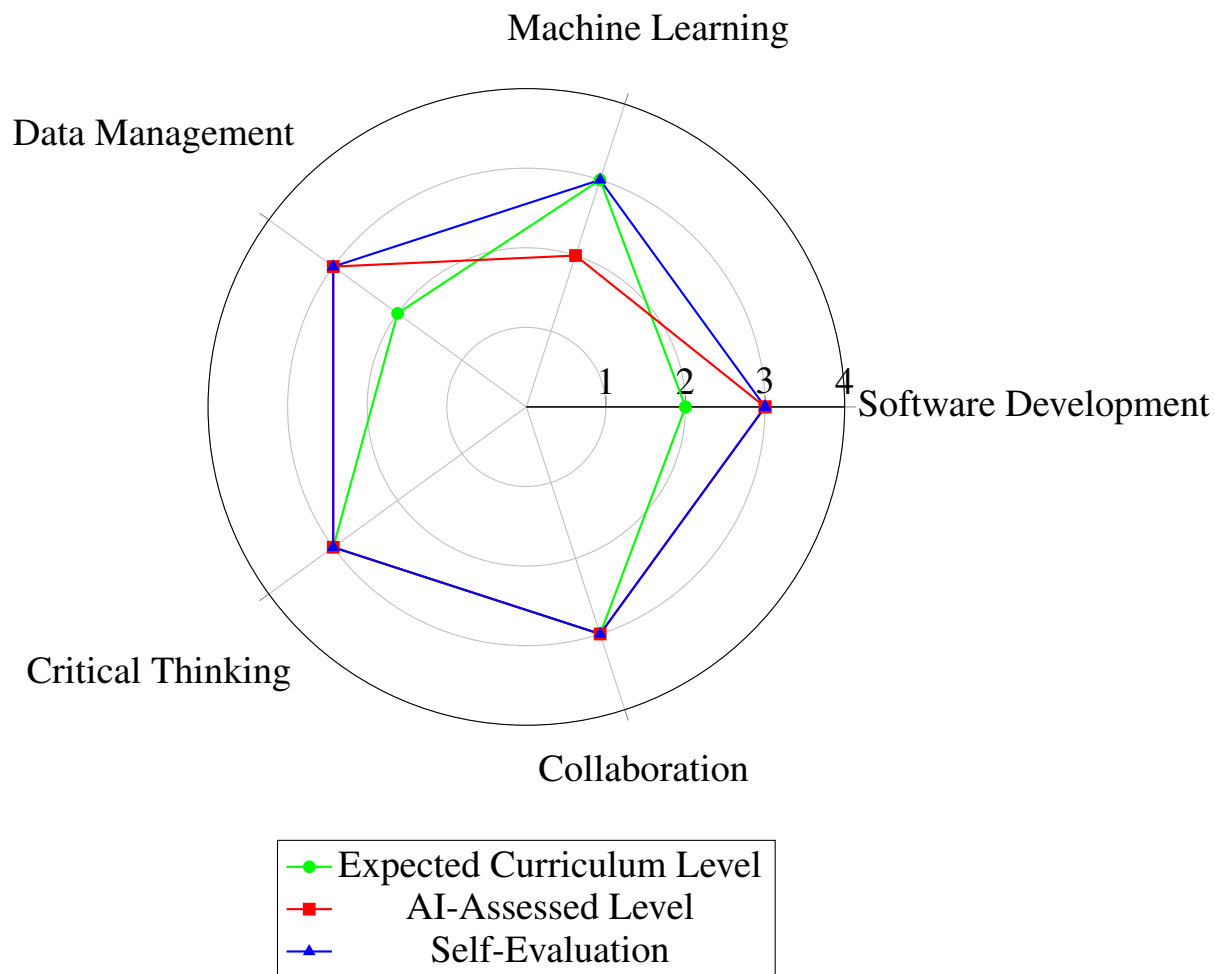


Figure 9.2: Radar chart comparing program targets, AI evaluation, and self-evaluation across five dimensions.

9.2.2 Reflection on Self-Evaluation Compared with AI Evaluation

Our self-evaluation is generally consistent with the AI assessment in recognizing strong performance in software engineering, data pipeline design, and technical communication. In particular, we agree that this project demonstrates structured architecture, reproducible experimentation, and clear traceability from problem definition to evaluation outputs.

The main difference is in how the Machine Learning dimension is interpreted. The AI rates this dimension lower because the final predictive performance is still limited, and we accept that result from an outcome perspective. However, from our perspective, this rating does not fully reflect the difficulty and novelty of the chosen problem. Our project focuses on prestimulus EEG behavior prediction in a CCD setting, which is a high-uncertainty task with substantial research and engineering risk. This type of work commonly requires multi-year depth in graduate-level research contexts, whereas this project was conducted at undergraduate level within a timeline of less than one year.

Because of that challenge, substantial project time was invested in foundational research engineering work before model optimization could be done effectively. This includes strict trial-label alignment, leakage-safe subject-independent splitting, quality checks for signal validity, and reproducible experiment tracking. These steps are essential for trustworthy ML experimentation, but they reduce the time available for broader architecture exploration and extensive hyperparameter optimization.

For this reason, our self-evaluation remains Level 3 across all dimensions, reflecting confidence in the rigor of the development and experimentation process, while still acknowledging that Machine Learning outcomes have not yet reached the desired performance level. In summary, the gap is more about model maturity than lack of technical capability or effort.

9.3 Advisor Evaluation

The third part focuses on the evaluation conducted by the project advisor. The advisor assesses the student's knowledge and skills and provides reasons for assigning each level.

9.3.1 Radar Chart Comparing Program Targets, AI Evaluation, Self-Evaluation, and Advisor Evaluation

This section is completed only after the first two parts are finalized. The advisor reads the previous sections and then conducts the third

evaluation. A radar chart should be included with four lines in the same figure: (1) the program's target levels, (2) the AI-based evaluation, (3) the student's self-evaluation, and (4) the advisor's evaluation. This figure provides a comprehensive comparison across all perspectives.

[To be completed by the advisor.]

9.3.2 Advisor's Comments on the Evaluation

This section summarizes the advisor's opinions and explanations regarding the evaluation. The advisor clarifies why each level was assigned and may comment on the student's strengths, progress, limitations, and future development directions.

[To be completed by the advisor.]

9.3.3 Student Reflection on the Advisor's Evaluation

This section presents the student's reflection on the advisor's assessment. The student may discuss points of agreement or disagreement, lessons learned from the advisor's perspective, and how the evaluation contributes to personal and professional development.

[To be completed by the authors.]

Chapter 10

Conclusion and Discussion

This chapter concludes the report by reflecting on the project outcomes, summarizing key challenges encountered during development and evaluation, and outlining directions for future improvement.

10.1 Reflection

This project set out to answer a constrained research question—whether prestimulus EEG alone can predict behavioral performance—and to deliver software that makes this question testable in a reproducible way. The main contribution is an end-to-end workflow that links (1) event-locked dataset construction for CCD-ITI windows, (2) standardized preprocessing and feature extraction for steady-state characteristics, (3) dual-target modeling (RT regression and correctness classification), and (4) consistent evaluation and reporting artifacts.

From a software engineering perspective, the outcome is a clearer separation between data ingestion, processing, modeling, and reporting. This reduces the risk that results depend on hidden notebook state and makes it easier to rerun experiments, compare configurations, and regenerate figures/tables for the final report.

From a research perspective, the journey was difficult: substantial effort was required before model training became stable and comparable. Even after overcoming pipeline and data-quality challenges, the final models still did not achieve strong predictive performance.

10.2 Challenges

The main challenges encountered are:

- **Trial and label alignment:** prestimulus prediction depends on strict alignment between events, ITI windows, and behavioral labels; small mistakes can invalidate comparisons.
- **Data quality and non-stationarity:** EEG recordings vary across subjects and sessions, and artifacts/noise can dominate unless preprocessing and sanity checks are systematic.
- **Class imbalance and instability for correctness:** several runs showed strong sensitivity but weak specificity, indicating prediction collapse toward one class despite acceptable overall accuracy.
- **Imbalance mitigation with limited gain:** class weighting, weighted sampling, and undersampling were tested to correct imbalance effects, but improvements were partial and did not deliver consistently reliable classification quality.
- **Weak RT explanatory power:** regression runs with MAE/MSE/Huber losses all produced near-zero R^2 , suggesting underfitting or insufficient predictive signal extraction in the current setup.
- **Reproducibility overhead:** making experiments repeatable requires extra engineering (configs, caching, logging, deterministic splits) beyond what is needed for a one-off notebook result.

10.3 Future Work

Future work should prioritize making the platform easier to extend and easier to use in real settings. Specific next steps address the observed limitations:

Platform Usability and Extensibility (Primary Priority)

- **Plugin-ready architecture:** Define clear interfaces for preprocessing, feature extraction, and modeling so new steps can be added without modifying core code. Provide reference implementations and documentation for each extension point.

- **User onboarding and documentation:** Write step-by-step tutorials, a user manual with screenshots, and documented example workflows for common tasks (loading BIDS data, running preprocessing, launching a model). Include troubleshooting guides.
- **Usability testing and feedback loops:** Conduct user testing sessions with domain researchers to identify confusing UI steps. Collect satisfaction surveys and iterate on the interface based on real feedback.
- **Operational clarity:** Improve error messages with actionable guidance, add run summaries showing what succeeded and what failed, and keep structured experiment logs (inputs, outputs, parameters, timing) for reproducible comparisons and failure auditing.

Model-Focused Improvements (Secondary Priority)

- **Fix class imbalance and evaluation metrics:** The correctness classifier showed 95% sensitivity but only ~10% specificity on error detection. Replace accuracy with balanced metrics (F1, MCC) during training and hyperparameter selection. Test threshold tuning and probability calibration post-hoc.
- **Improve RT regression capacity:** Current MAE/MSE/Huber experiments all produced R^2 near zero. Explore richer feature representations (time-frequency, connectivity) and stronger regularization schedules to move beyond mean-prediction behavior.

Reference

Bibliography

- [1] B. Aristimunha, D. Truong, P. Guetschel, S. Y. Shirazi, I. Guyon, A. R. Franco, M. P. Milham, A. Dotan, S. Makeig, A. Gramfort, J.-R. King, M.-C. Corsi, P. A. Valdés-Sosa, A. Majumdar, A. Evans, T. J. Sejnowski, O. Shriki, S. Chevallier, and A. Delorme, “Eeg foundation challenge: From cross-task to cross-subject eeg decoding,” *NeurIPS 2025 Competition Proposal* (preprint), 2025.
- [2] S. Y. Shirazi, A. Franco, M. S. Hoffmann, N. B. Esper, D. Truong, A. Delorme, M. P. Milham, and S. Makeig, “Hbn-eeg: The fair implementation of the healthy brain network (hbn) electroencephalography dataset,” *bioRxiv*, 2024.
- [3] L. M. Alexander, J. Escalera, L. Ai, C. Andreotti, K. Febre, A. Mangone, N. Vega-Potler, N. Langer, A. Alexander, M. Kovacs, S. Litke, B. O’Hagan, J. Andersen, B. Bronstein, A. Bui, M. Bushey, H. Butler, V. Castagna, N. Camacho, E. Chan, D. Citera, J. Clucas, S. Cohen, S. Dufek, M. Eaves, B. Fradera, J. Gardner, N. Grant-Villegas, G. Green, C. Gregory, E. Hart, S. Harris, M. Horton, D. Kahn, K. Kabotyanski, B. Karmel, S. P. Kelly, K. Kleinman, B. Koo, E. Kramer, E. Lennon, C. Lord, G. Mantello, A. Margolis, K. R. Merikangas, J. Milham, G. Minniti, R. Neuhaus, A. Levine, Y. Osman, L. C. Parra, K. R. Pugh, A. Racanello, A. Restrepo, T. Saltzman, B. Septimus, R. Tobe, R. Waltz, A. Williams, A. Yeo, F. X. Castellanos, A. Klein, T. Paus, B. L. Leventhal, R. C. Craddock, H. S. Koplewicz, and M. P. Milham, “An open resource for transdiagnostic research in pediatric mental health and learning disorders,” *Scientific Data*, vol. 4, no. 170181, 2017.
- [4] N. Langer, E. J. Ho, L. M. Alexander, H. Y. Xu, R. K. Jozanovic, S. Henin, A. Petroni, S. Cohen, E. T. Marcelle, L. C. Parra, M. P. Milham, and S. P. Kelly, “A resource for assessing information processing

in the developing brain using eeg and eye tracking,” *Scientific Data*, vol. 4, no. 170040, 2017.

Appendix A

Appendix A: Reproducibility Summary

This appendix summarizes the fixed workflow and evaluation protocol used throughout the project.

A.1 Data and Task Scope

- Dataset context: HBN-EEG, Contrast Change Detection (CCD) task.
- Modeling window: fixed 2-second prestimulus inter-trial interval (ITI).
- Targets: reaction time (regression) and trial correctness (binary classification).

A.2 Fixed Processing Stages

The operational pipeline used in this report is:

1. Ingestion (load runs, events, and metadata)
2. Trial construction (ITI extraction and label attachment)
3. Preprocessing (filtering/normalization/artifact mitigation)
4. Verification (SNR and related signal checks)
5. Feature construction
6. Modeling (regression and classification)
7. Evaluation and artifact export

A.3 Evaluation Metrics

- Regression: MAE, RMSE, R^2 , Pearson correlation.

- Classification: accuracy, balanced accuracy, sensitivity, specificity.

Appendix B

Appendix B: Detailed Result Tables

This appendix records detailed run summaries from the provided W&B exports.

B.1 Classification Runs

Run	Ch	Strategy	WCIs	Smooth	Acc	Bal Acc	Sens	Spec	ROC AUC	F1 Bin
allch_none	128	none	false	0.00	0.729	0.501	0.993	0.009	0.577	0.843
allch_base	128	class_weight	true	0.00	0.604	0.544	0.674	0.414	0.566	0.713
allch_fastdecay	128	class_weight	true	0.05	0.592	0.544	0.648	0.439	0.569	0.699
allch_smooth	128	class_weight	true	0.05	0.593	0.543	0.650	0.437	0.569	0.700
allch_sampler	128	weighted_sampler	false	0.00	0.629	0.543	0.729	0.358	0.573	0.742
allch_undersample	128	undersample	false	0.00	0.652	0.537	0.787	0.287	0.582	0.768
limitch_base	13	weighted_sampler	false	0.00	0.660	0.543	0.796	0.290	0.560	0.774

Table B.1: Detailed classification run comparison.

Column notes: Ch = number of EEG channels, WCIs = class-weighting enabled, Smooth = label smoothing value.

B.2 Regression Runs

Run	Loss	MAE	RMSE	R ²	Pearson	Expl.Var	Pred Mean	True Mean
eegnet_mae	mae	0.2876	0.3743	0.0063	0.0840	0.0069	1.6114	1.6019
eegnet_mse	mse	0.2883	0.3749	0.0030	0.0562	0.0030	1.6035	1.6019
eegnet_huber	huber	0.2874	0.3740	0.0079	0.0903	0.0081	1.6070	1.6019

Table B.2: Detailed regression run comparison.

Column notes: Expl.Var = explained variance, Pred Mean = test/y_pred_mean, True Mean = test/y_true_mean.